3. Attention

Transformer

[Transformer]

Attention Is All You Need, NeurIPS 2017

- **A short review about the "Transformer"** for the understanding of the Vision transformer
  - It is essential to figure out what attention is

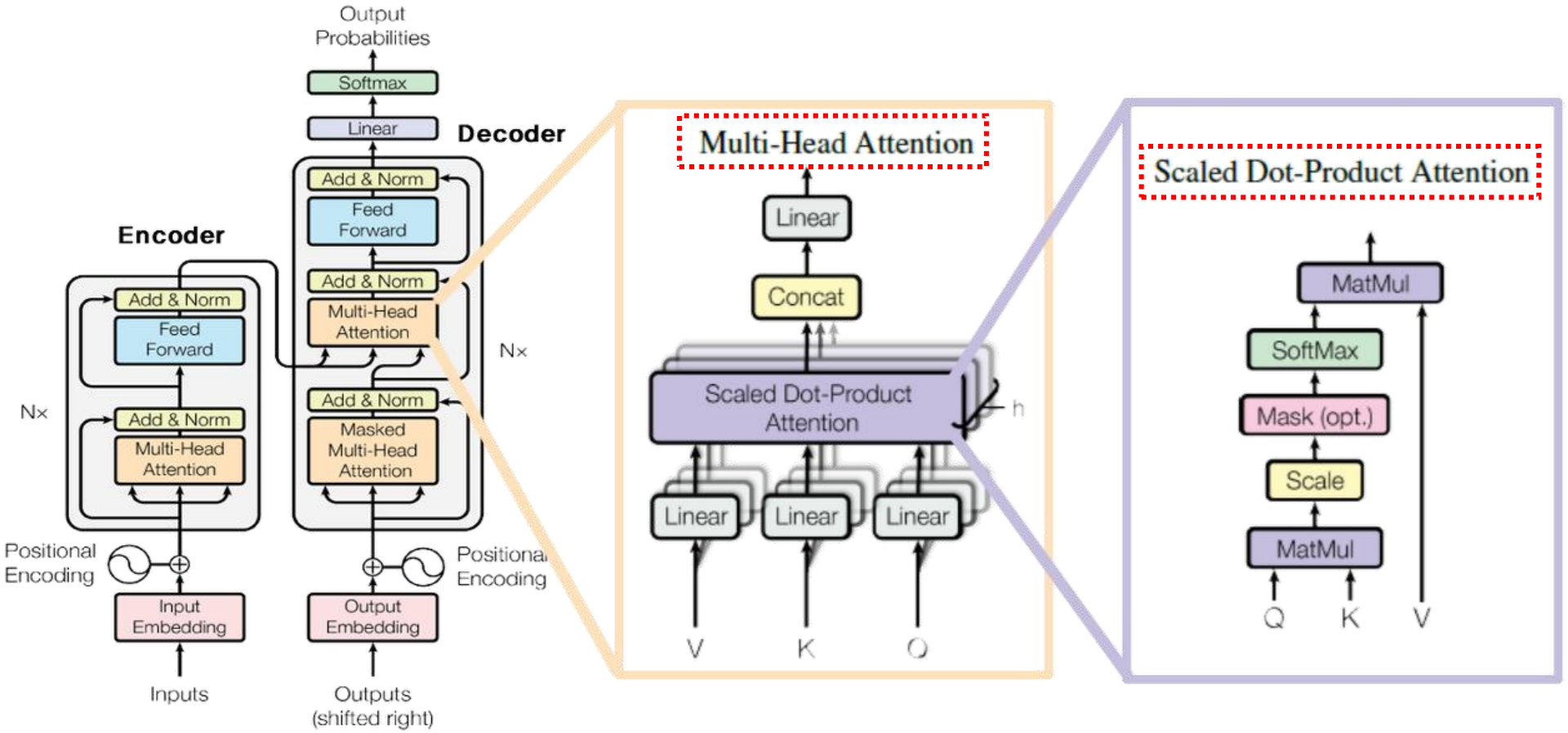
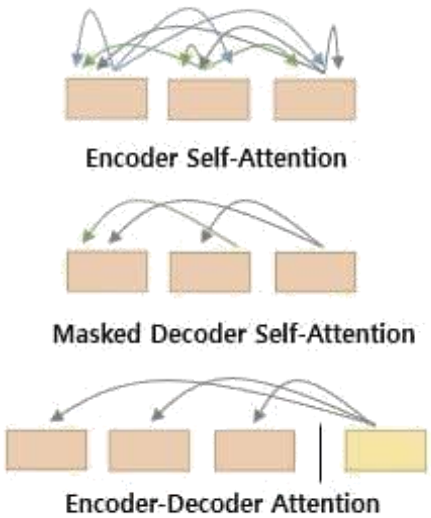


Figure 1: The Transformer - model architecture.

Attentions

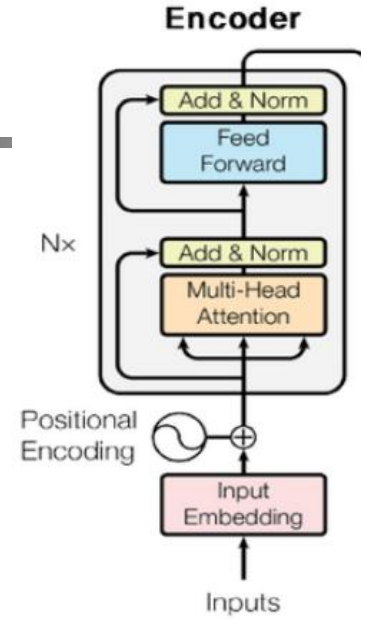


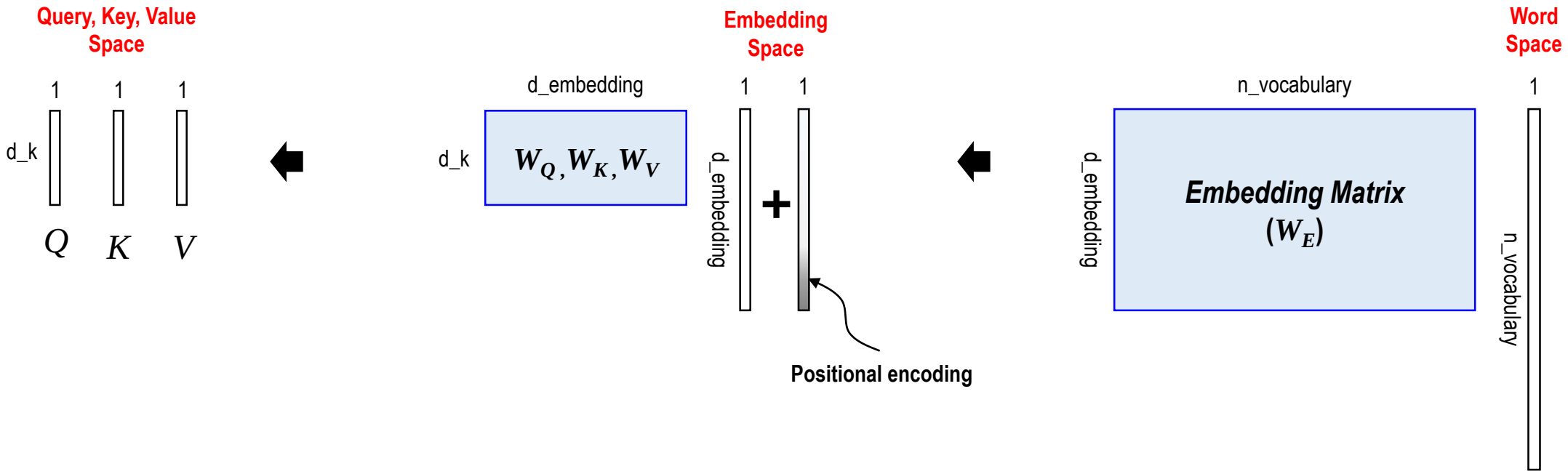
https://hyunsooworld.tistory.com/entry/%EC%B5%9C%EB%8C%80%ED%95%9C-%EC%89%BD%EA%B2%8C-%EC%84%A4%EB%AA%85%ED%95%9C-%EB%85%BC%EB%AC%B8%EB%A6%AC%EB%B7%B0-Attention-Is-All-You-NeedTransformer-%EB%85%BC%EB%AC%B8

https://blog-ko.superb-ai.com/what-is-the-transformer-model/

Encoder (1/2)

- **Input (word) embedding**
  - Process of mapping each discrete token into a dense, continuous vector space
- **Linear projection**
  - Query (Q), Key (K), and Value (V) vectors are generated by linearly projecting the input embeddings using learnable weight matrices (W_Q, W_K, W_V)





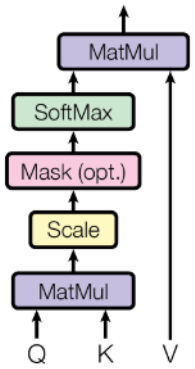
: Learnable parameters

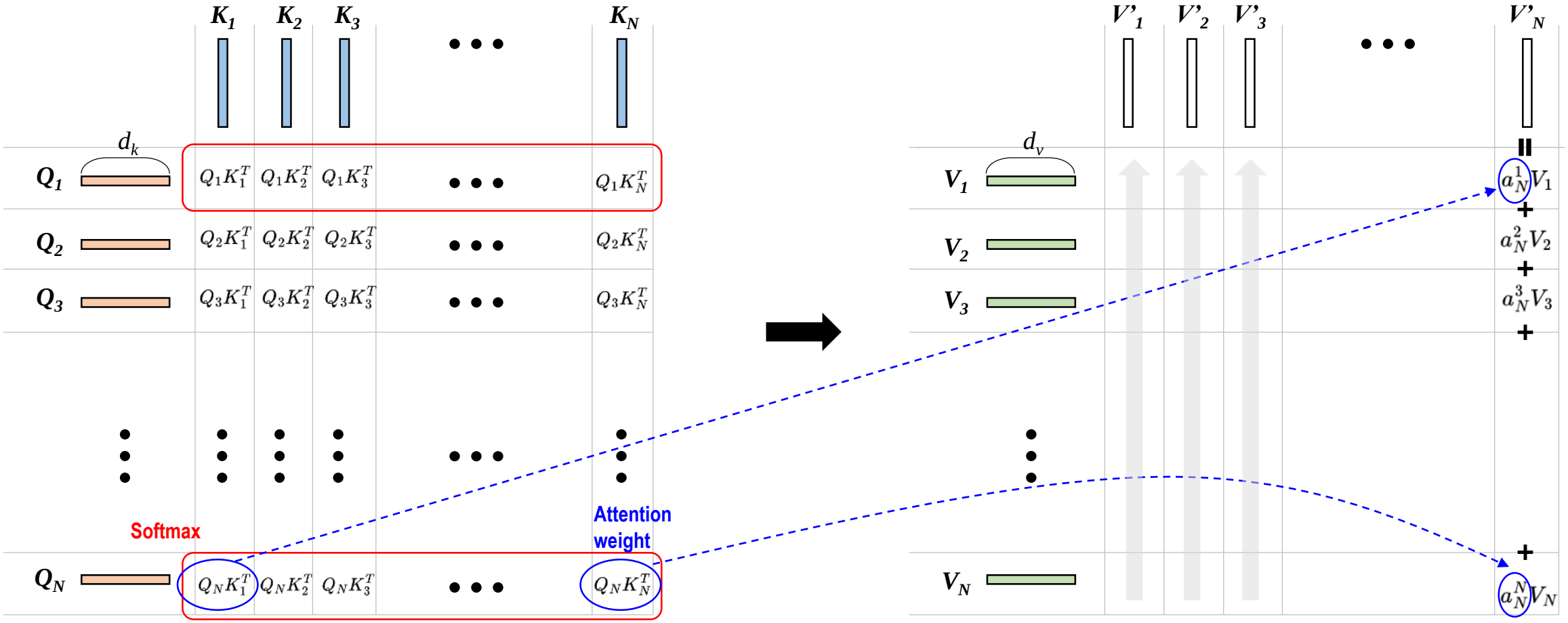
Encoder (2/2)

- **Scaled Dot-Product Attention**
  - Attention weights derived from the dot-product between Q and K
  - Attention map grows quadratically with the sequence length (N)

$$\frac{e^{z_i}}{\sum_{j=1}^{K} e^{z_j}}$$

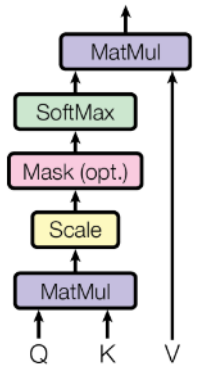
$$\text{Attention}(Q, K, V) = \text{softmax}(\frac{QK^T}{\sqrt{d_k}})V$$





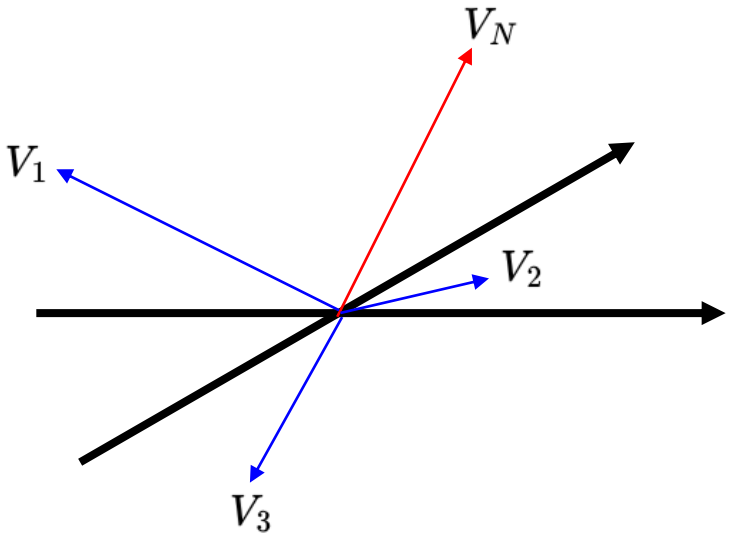
Scaled Dot-Product Attention

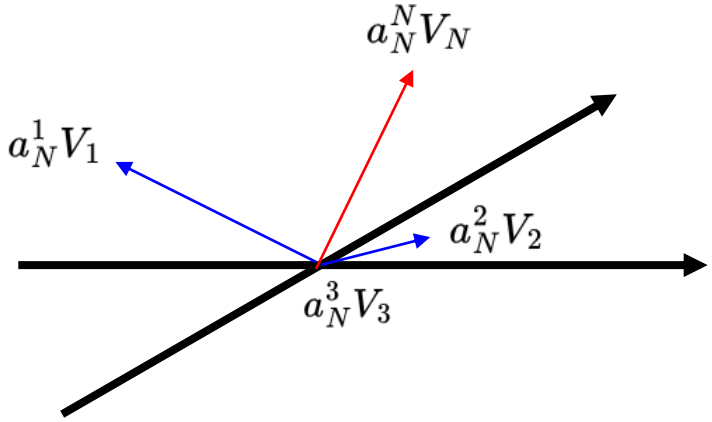
Attention (1/2)



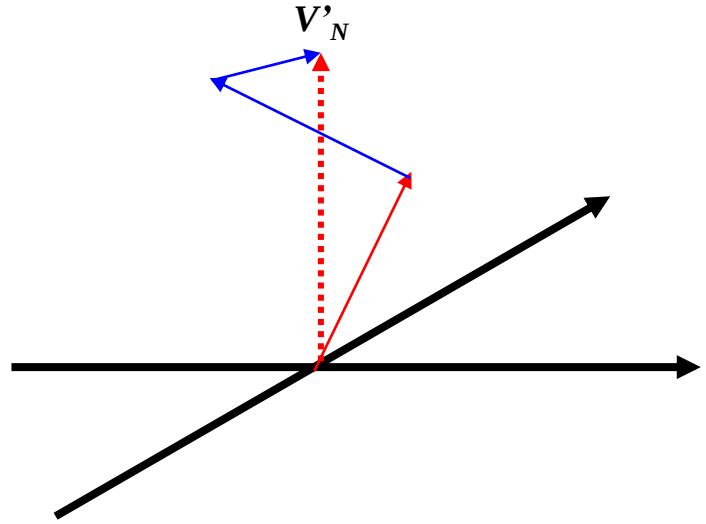
- **Meaning of the 'Scaled Dot-Product Attention'**
  - Weighted sum of the value vectors

Value Space





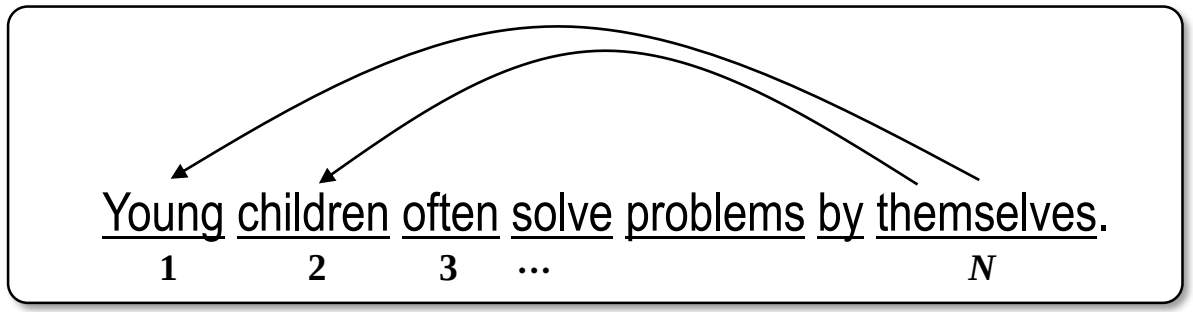
[Attention weighting]

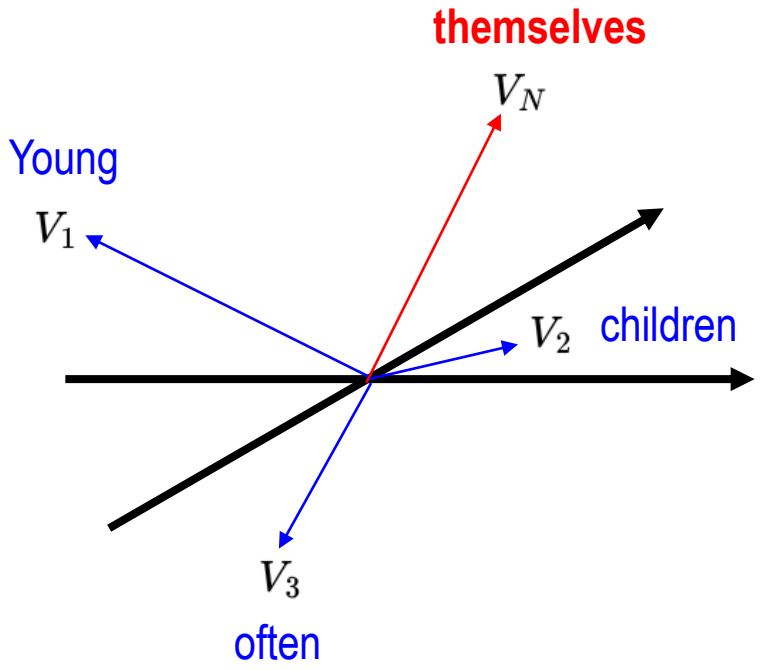


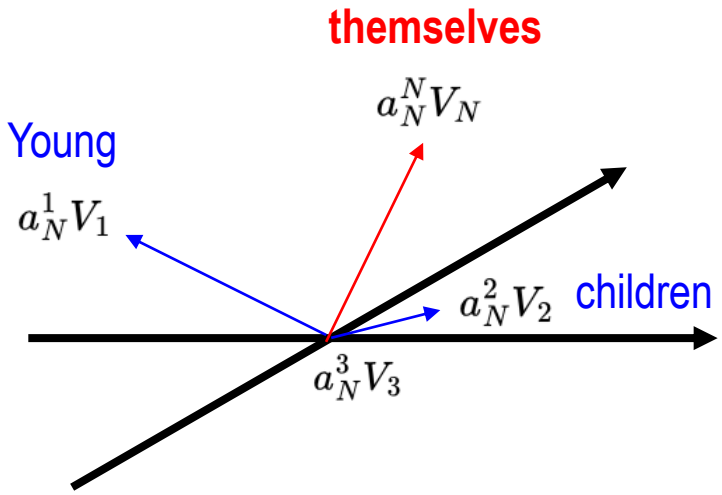
[Weighted sum]

Attention (2/2)

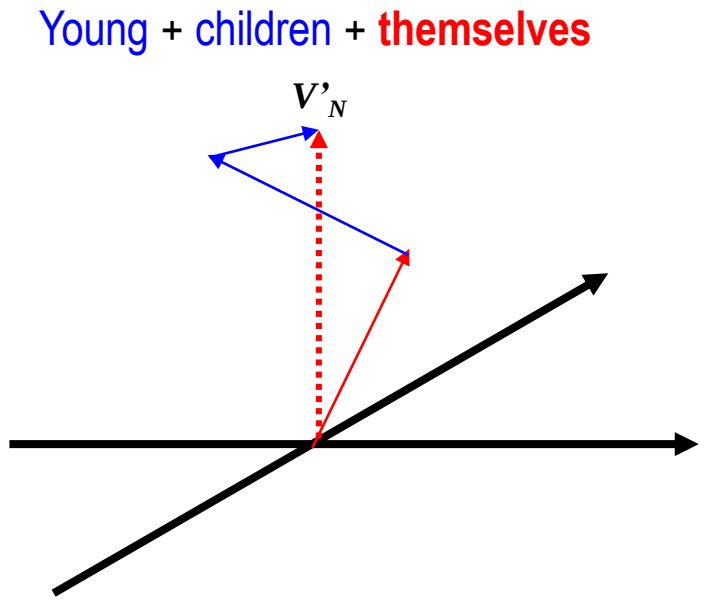
- **Global attention**
  - Every token can attend to every other token (Global receptive field)







[Attention weighting]



[Weighted sum]

Vision Transformer (ViT)

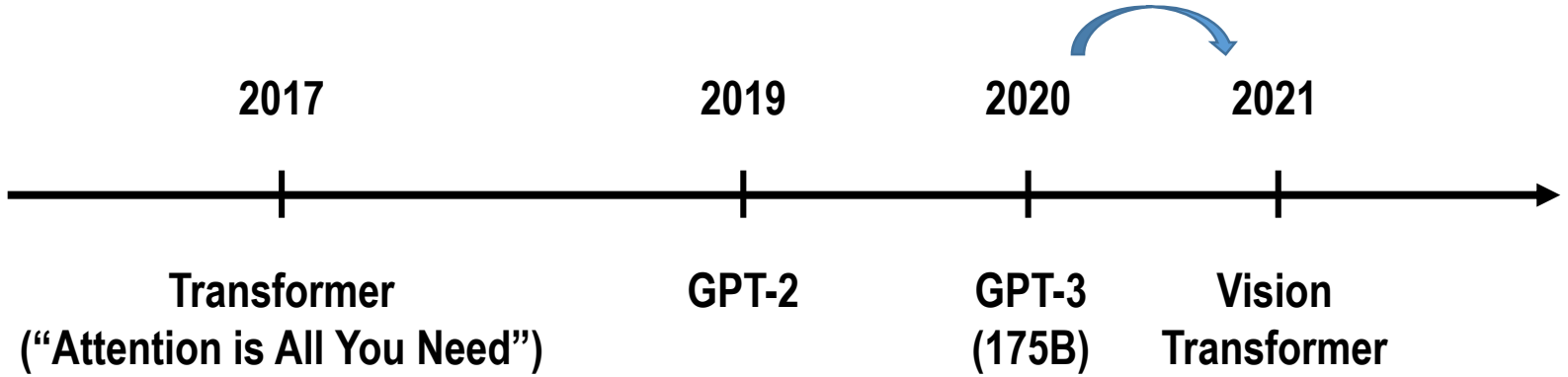
[Vision Transformer]

- **Limitations of CNNs**
  - Performance saturation
    - Gradual diminishing rate of performance improvement when the model size or data scale was increased
  - What lies beyond the *Inductive bias of CNN ?
    - Locality : learns local patterns only → struggles with long-range dependencies
    - Translation equivalence

* The set of assumptions a learning algorithm uses to make predictions about unseen data

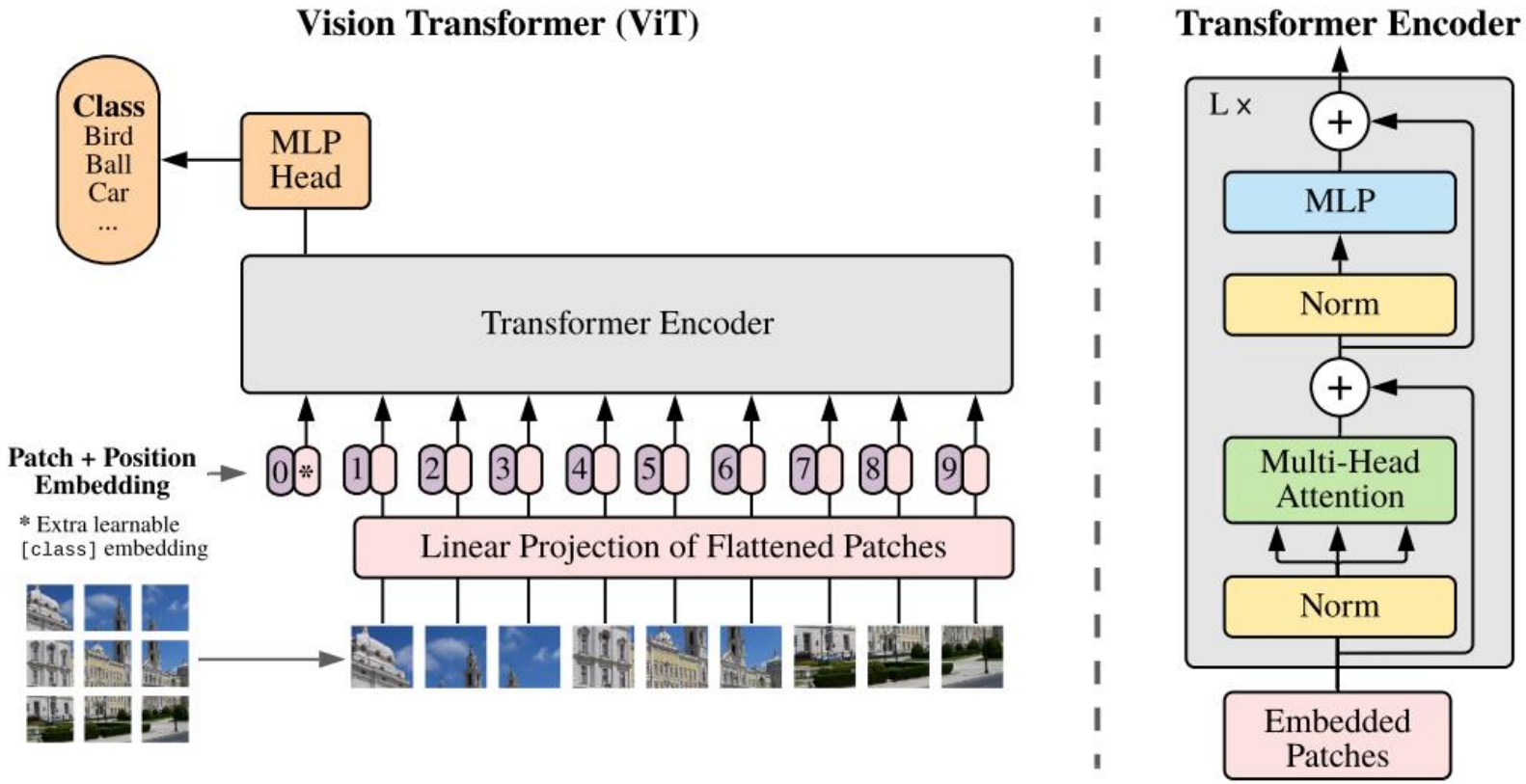
- **Rise of Transformers**
  - Transformers achieved huge success in NLP by learning global relationships
  - Question:

    "Can self-attention replace convolution in vision tasks?"



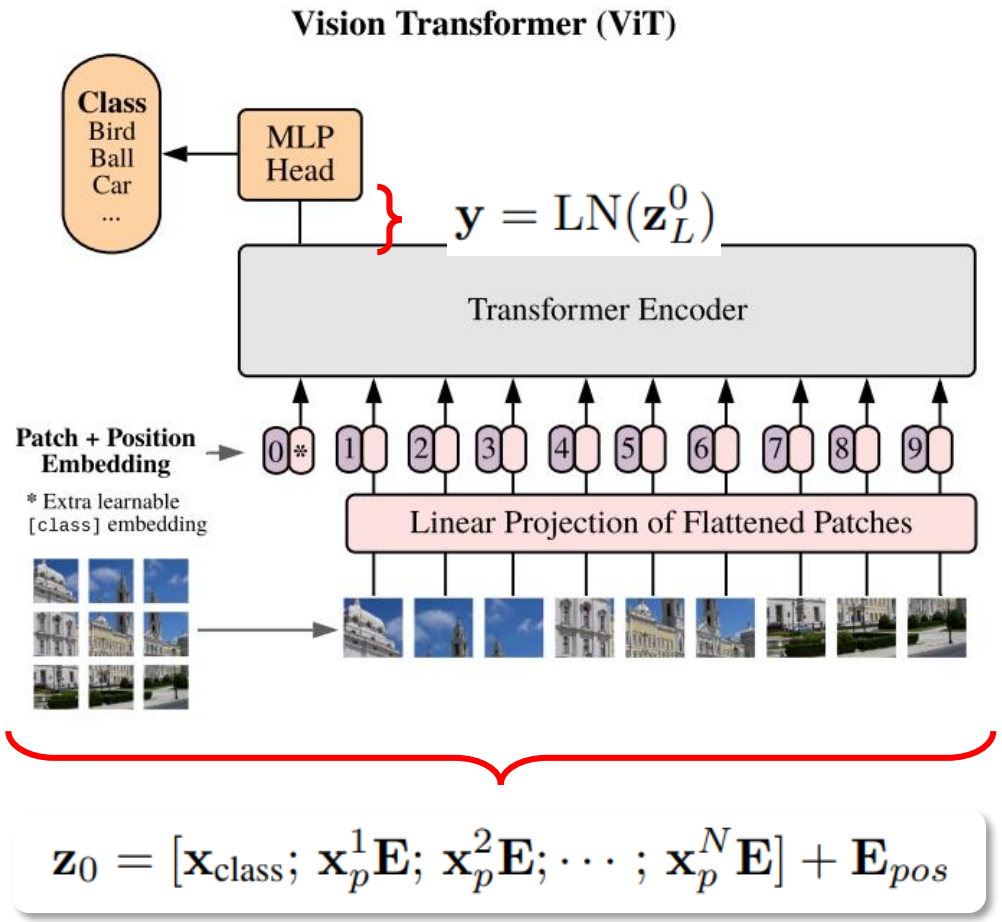
Architecture (1/2)

- Treat an image as **a sequence of patches**
- Use the output of [CLS] token for **classification**
- Feed position-embedded patches into a **Transformer Encoder**
- **MLP Head** maps the final [CLS] token into task-specific output logits for prediction (e.g., classification)
  - Attention-based global summary of all image patches



Architecture (2/2)

- **Details on the architecture**



Layer Normalization

Skip connection

Learnable parameters

$$\mathbf{z}_\ell = \mathrm{MLP}(\mathrm{LN}(\mathbf{z}'_\ell)) + \mathbf{z}'_\ell$$

$$\mathbf{z}'_\ell = \mathrm{MSA}(\mathrm{LN}(\mathbf{z}_{\ell-1})) + \mathbf{z}_{\ell-1},$$

$$\mathrm{MSA}(\mathbf{z}) = [\mathrm{SA}_1(z); \mathrm{SA}_2(z); \cdots; \mathrm{SA}_k(z)]\, \mathbf{U}_{msa}$$

$$[\mathbf{q}, \mathbf{k}, \mathbf{v}] = \mathbf{z}\mathbf{U}_{qkv}$$

$$A = \mathrm{softmax}\left(\mathbf{q}\mathbf{k}^\top / \sqrt{D_h}\right)$$

$$\mathrm{SA}(\mathbf{z}) = A\mathbf{v}\,.$$

Similarity measure (i.e., attention)

Patch embedding

- **Image → Flattened patch sequence**



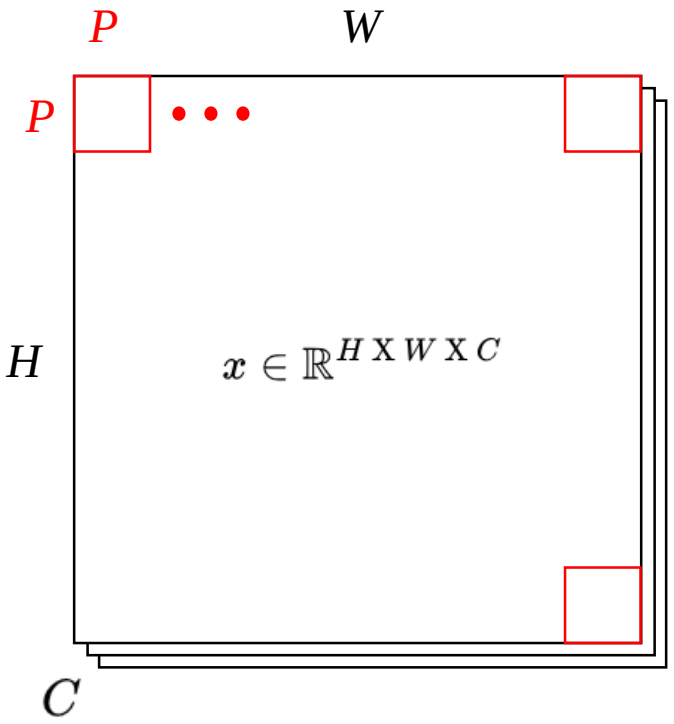
of pixels :

P^2C

of patches :

$N = \frac{HW}{P^2}$

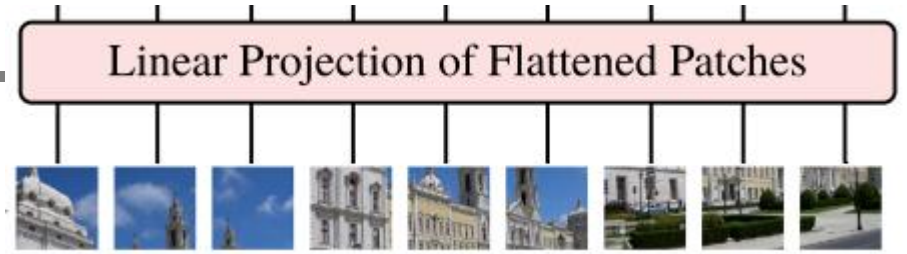
$x_p \in \mathbb{R}^{N \times (P^2C)}$



$x \in \mathbb{R}^{H \times W \times C}$

Linear projection

- **Linear projection of flattened patches**



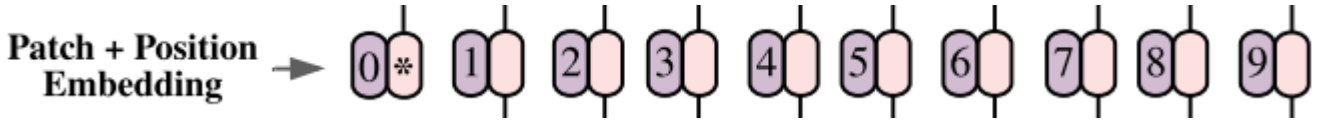
D P^2C D

N $x_p^1\mathbf{E};\ x_p^1\mathbf{E};,\cdots;\ x_p^N\mathbf{E}$ N $x_p \in \mathbb{R}^{N\ \mathrm{X}\ (P^2C)}$ P^2C $\mathbf{E}$

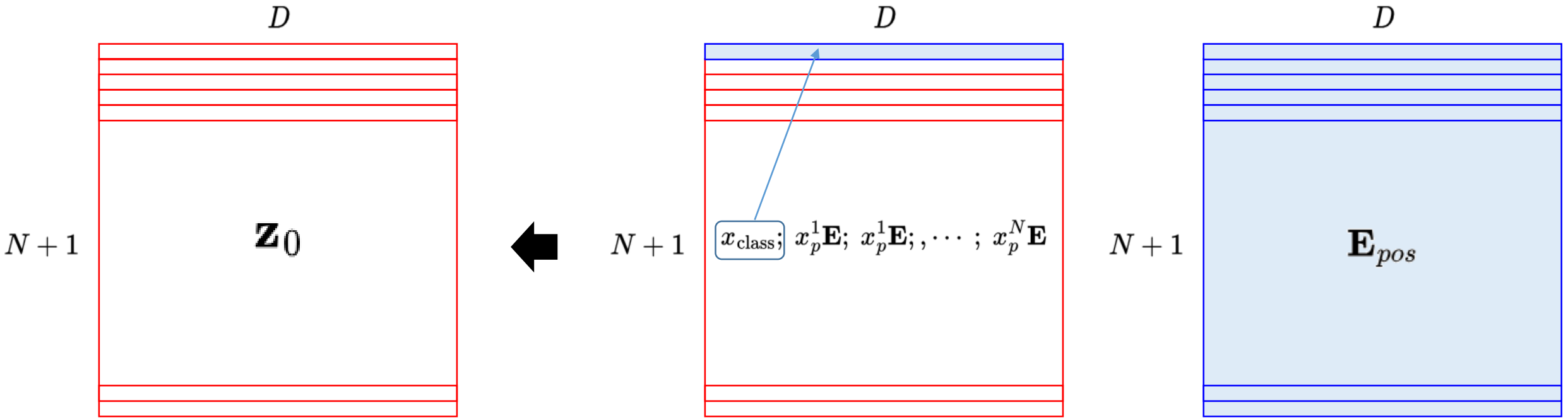
: Learnable parameters

Position embedding

- **Patch + Position embedding**
  - Concatenation of a learnable class token to the patch embedding
  - Element-wise addition of learnable position embedding and patch embedding



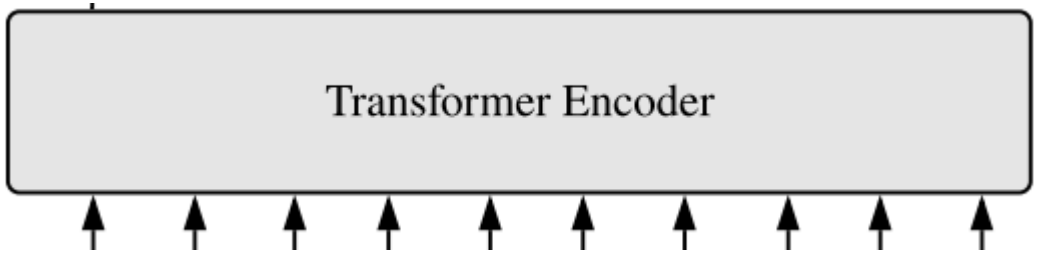
$$\mathbf{z}_0 = [\mathbf{x}_{\text{class}};\ \mathbf{x}_p^1\mathbf{E};\ \mathbf{x}_p^2\mathbf{E};\ \cdots\ ;\ \mathbf{x}_p^N\mathbf{E}] + \mathbf{E}_{pos}$$



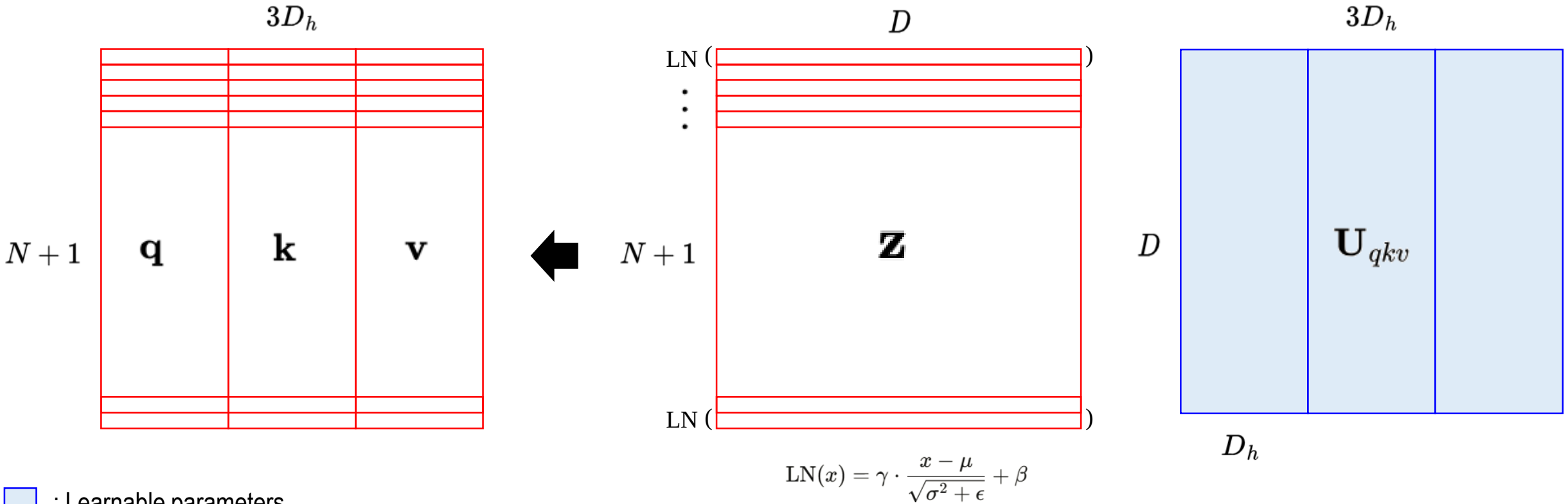
: Learnable parameters

Query / Key / Value

- **Q, K, V computation**
  - Linear projections from the input embeddings
  - Splitting the projected features into **q, k, v** of multiple heads



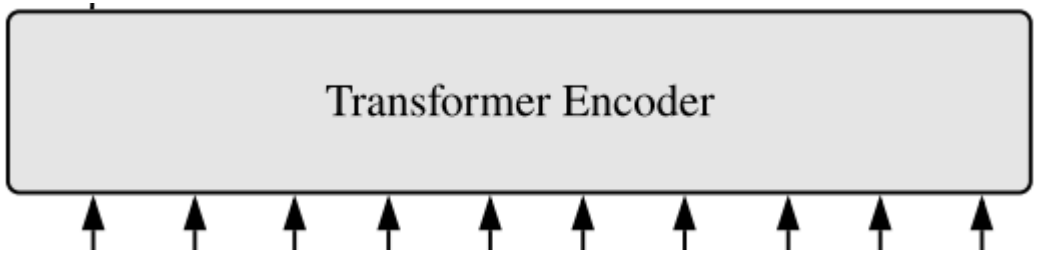
$$[\mathbf{q}, \mathbf{k}, \mathbf{v}] = \mathbf{z}\mathbf{U}_{qkv}$$



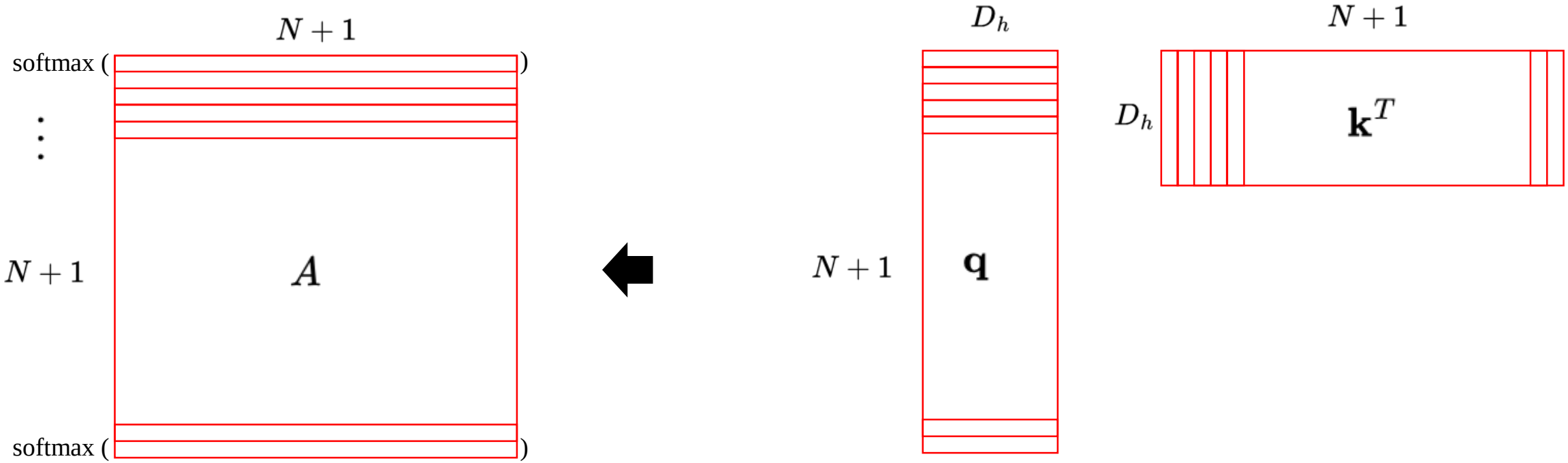
: Learnable parameters

Attention among patches

- **Self-attention map**
  - Attention among patches based on **q** and **k**

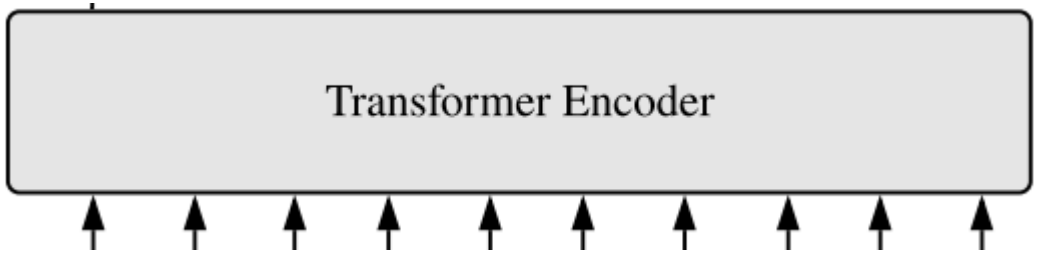


$$A = \text{softmax}\left(\mathbf{q}\mathbf{k}^\top / \sqrt{D_h}\right)$$

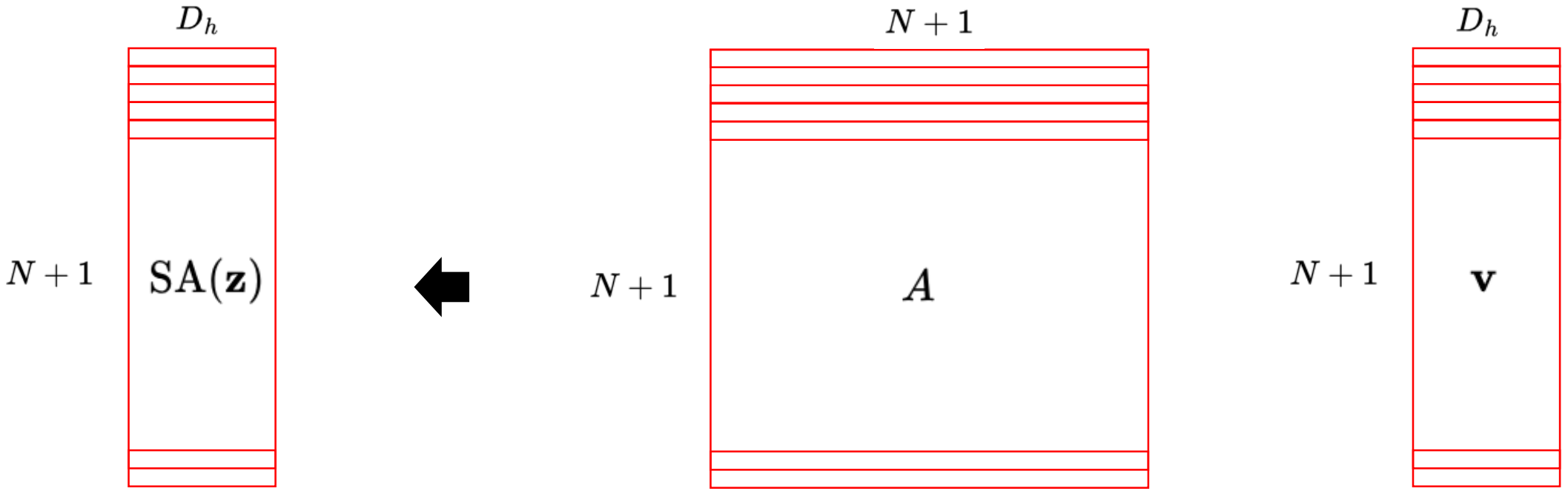


Self-attention

- **SA (Self-attention)**
  - Attention-wise weighted sum of value vectors

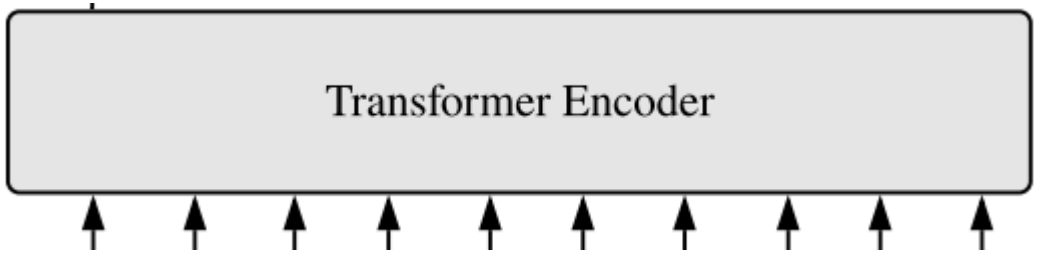


$$\mathrm{SA}(\mathbf{z}) = A\mathbf{v}$$

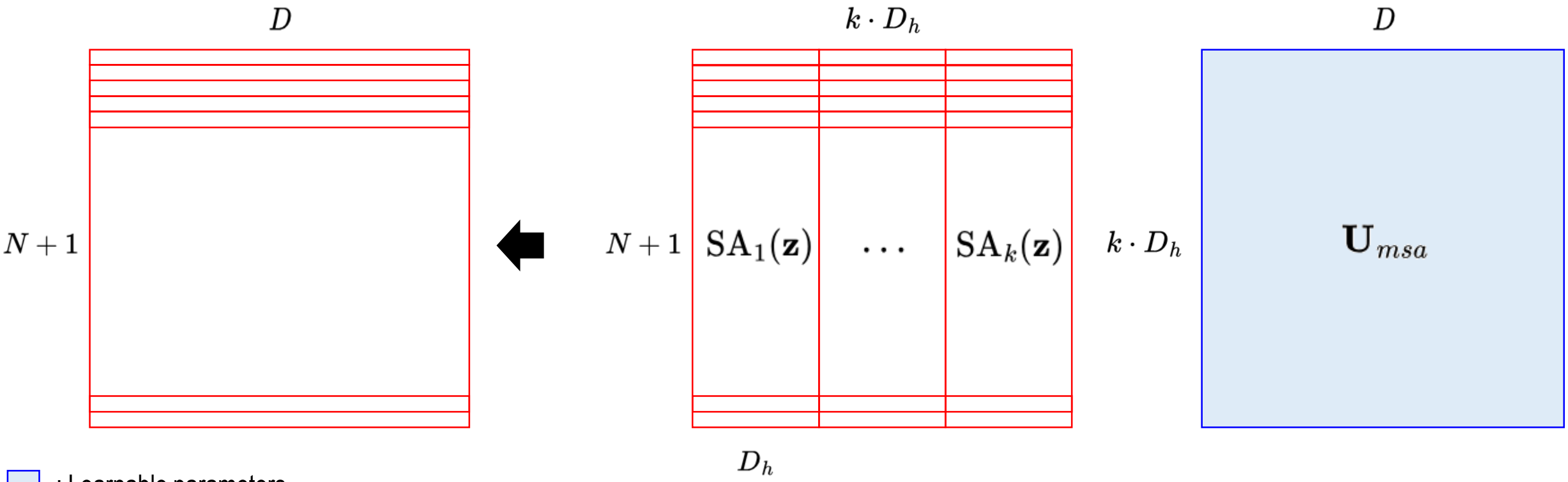


Multihead self-attenstion

- **MSA (Multihead self-attenstion)**
  - Concatenation of SAs from all heads
  - Linear projection from the concatenated SAs



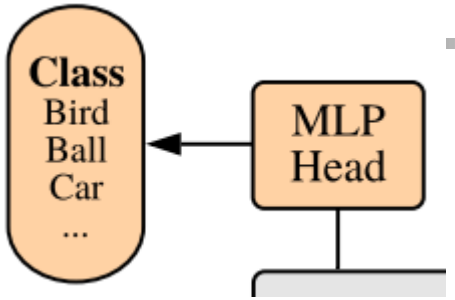
$$\mathrm{MSA}(\mathbf{z}) = [\mathrm{SA}_1(z); \mathrm{SA}_2(z); \cdots; \mathrm{SA}_k(z)]\, \mathbf{U}_{msa}$$



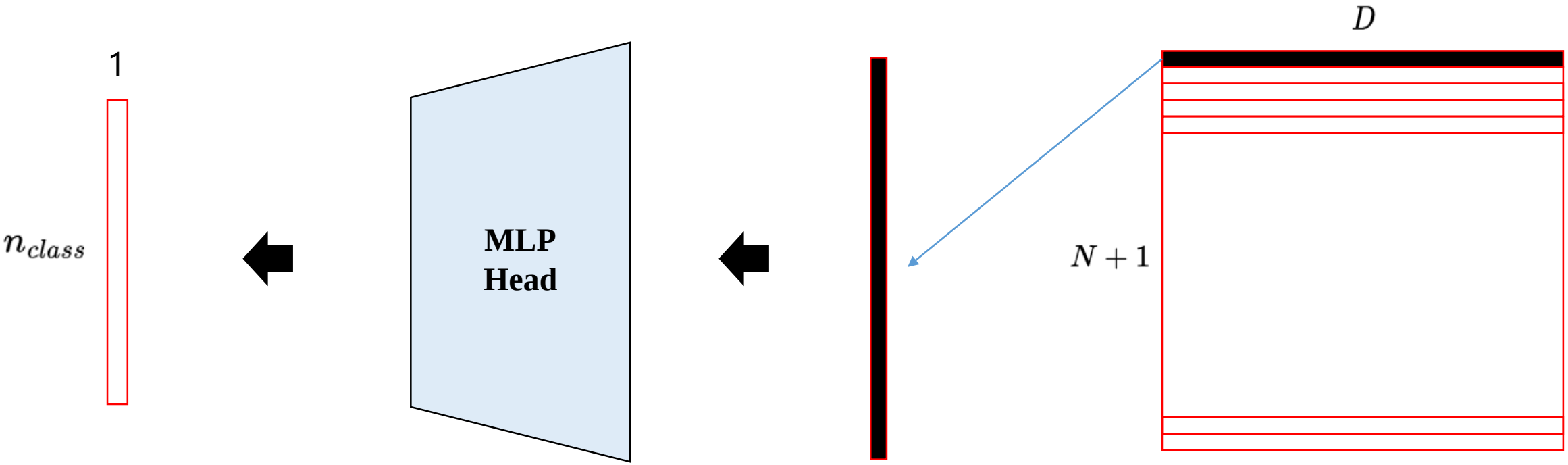
: Learnable parameters

Classification

- **MLP Head**
  - [CLS] token
    - aggregates global information by attending to all patch tokens
    - provides a fixed-size feature vector for a simple MLP head
  - Generates logits for softmax



$$\mathbf{y} = \mathrm{LN}(\mathbf{z}_L^0)$$



: Learnable parameters

Experiments (1/2)

- **Scaling up the dataset size**
  - ViT → Dramatic performance improvement
  - CNN → Saturation

Big Transfer (BiT): General Visual Representation Learning, CVPR 2020

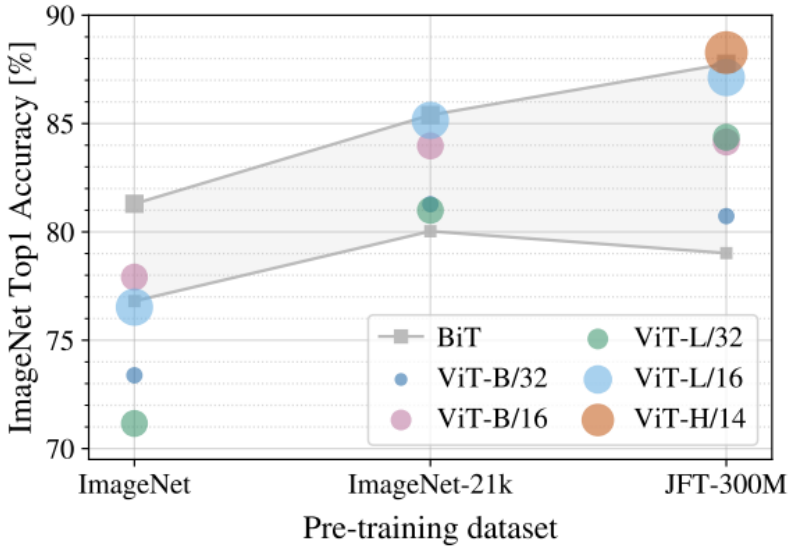


Figure 3: Transfer to ImageNet. While large ViT models perform worse than BiT ResNets (shaded area) when pre-trained on small datasets, they shine when pre-trained on larger datasets. Similarly, larger ViT variants overtake smaller ones as the dataset grows.

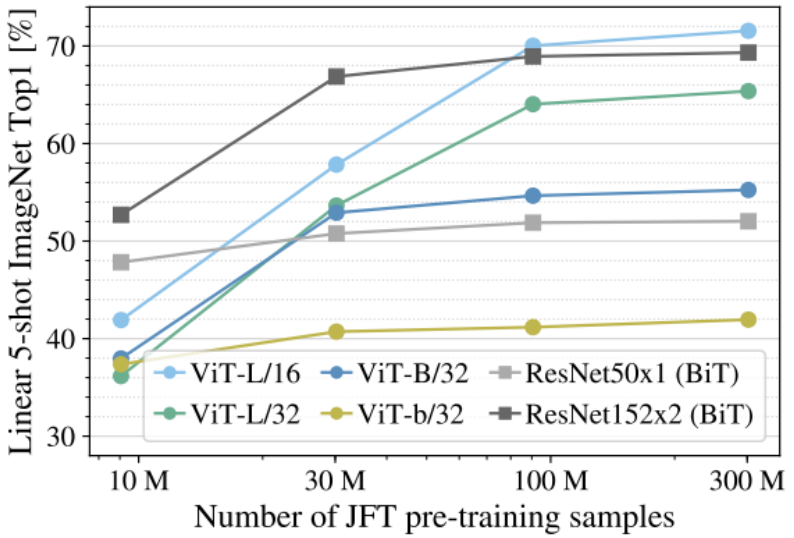


Figure 4: Linear few-shot evaluation on ImageNet versus pre-training size. ResNets perform better with smaller pre-training datasets but plateau sooner than ViT, which performs better with larger pre-training. ViT-b is ViT-B with all hidden dimensions halved.

(· / patch size)	Heads	Depth
ViT-Tiny (Ti/16)	3	12
ViT-Small (S/16)	6	12
ViT-Base (B/16)	12	12
ViT-Large (L/16)	16	24
ViT-Huge (H/14)	16	32

Experiments (2/2)

- **Positional encoding**
  - implicitly learns 2D spatial information using only 1D positional encoding
- **Attention mechanism**
  - integrates information from the entire image

Input size
224 x 224

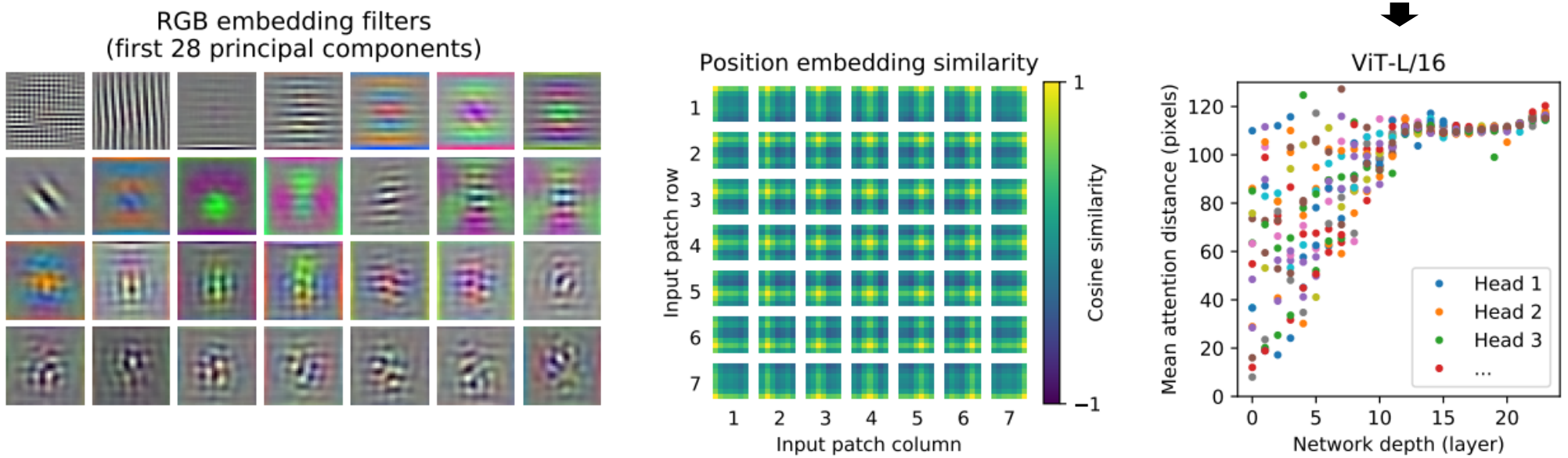


Figure 7: **Left:** Filters of the initial linear embedding of RGB values of ViT-L/32. **Center:** Similarity of position embeddings of ViT-L/32. Tiles show the cosine similarity between the position embedding of the patch with the indicated row and column and the position embeddings of all other patches. **Right:** Size of attended area by head and network depth. Each dot shows the mean attention distance across images for one of 16 heads at one layer. See Appendix D.7 for details.

Summary

- **Strengths**
  - Learns global patterns effectively
  - Simple computation blocks (matrix ops)
    → Hardware-friendly
  - Strong performance on large-scale tasks
    → Vision scaling

- **Limitations**
  - Requires very large datasets
    → prone to overfitting on small datasets

Dataset	#Images	#Classes	Public	Usage in ViT
ImageNet-1K	~1.28M	1,000	Yes	Fine-tuning / Baseline
ImageNet-21K	~14M	~21,000	Partially (restricted)	Pre-training
JFT-300M	~300M	~18,000	No (Google internal)	Large-scale Pre-training

  - Self-attention is computationally expensive, i.e., $O(N^2)$

ViT Variants

Post-ViTs

- All about **making ViT practical** — less data, less compute, more tasks, and more devices

Research Direction	Model	Outcome
Data Efficiency	**DeiT**, LV-ViT (2020)	Works well even on ImageNet-1K (Distillation → learns from CNN teacher)
Scalability & Hierarchy	**Swin** (2021)	Becomes usable beyond classification (Shifted Window Attention → scalable O(N))
Hardware & Memory Efficiency	EfficientFormer / MobileViT (2022)	Transformers move to edge/mobile AI (CNN–attention hybrid designs)
Cross-modal Unification	**CLIP**, ALIGN (2021)	Vision becomes compatible with language
Self-supervised representation	**DINO** (2021)	Rich, transferable, and interpretable visual representations without labels
CNN Reinvention	ConvNeXt (2022)	CNN and Transformer co-evolve (Renewed interest in next-generation CNNs)

[DeiT]

Training data-efficient image transformers & distillation through attention, 2020

- **Motivation**
  - Vision Transformer (ViT)
    - requires very large datasets, e.g., original paper uses JFT-300M (300M+ images)
  - Real-world constraint
    - Most environments only have ImageNet-1K (1.2M images) or less
- **Goal**
  - Data-efficient training strategy for Transformers
  - Train ViT from scratch using only ImageNet-1K based on knowledge distillation
    - No external data, no labels from large-scale datasets

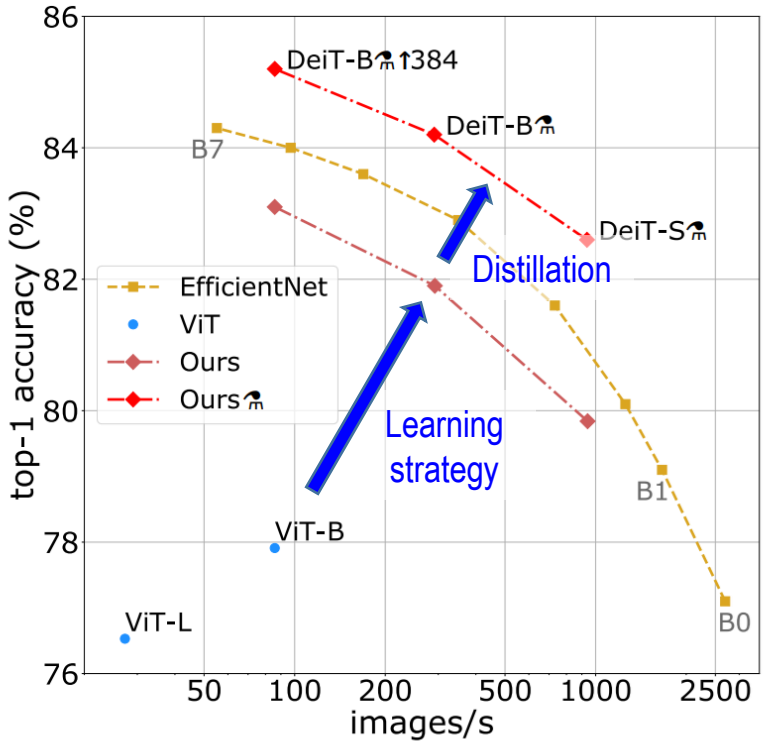
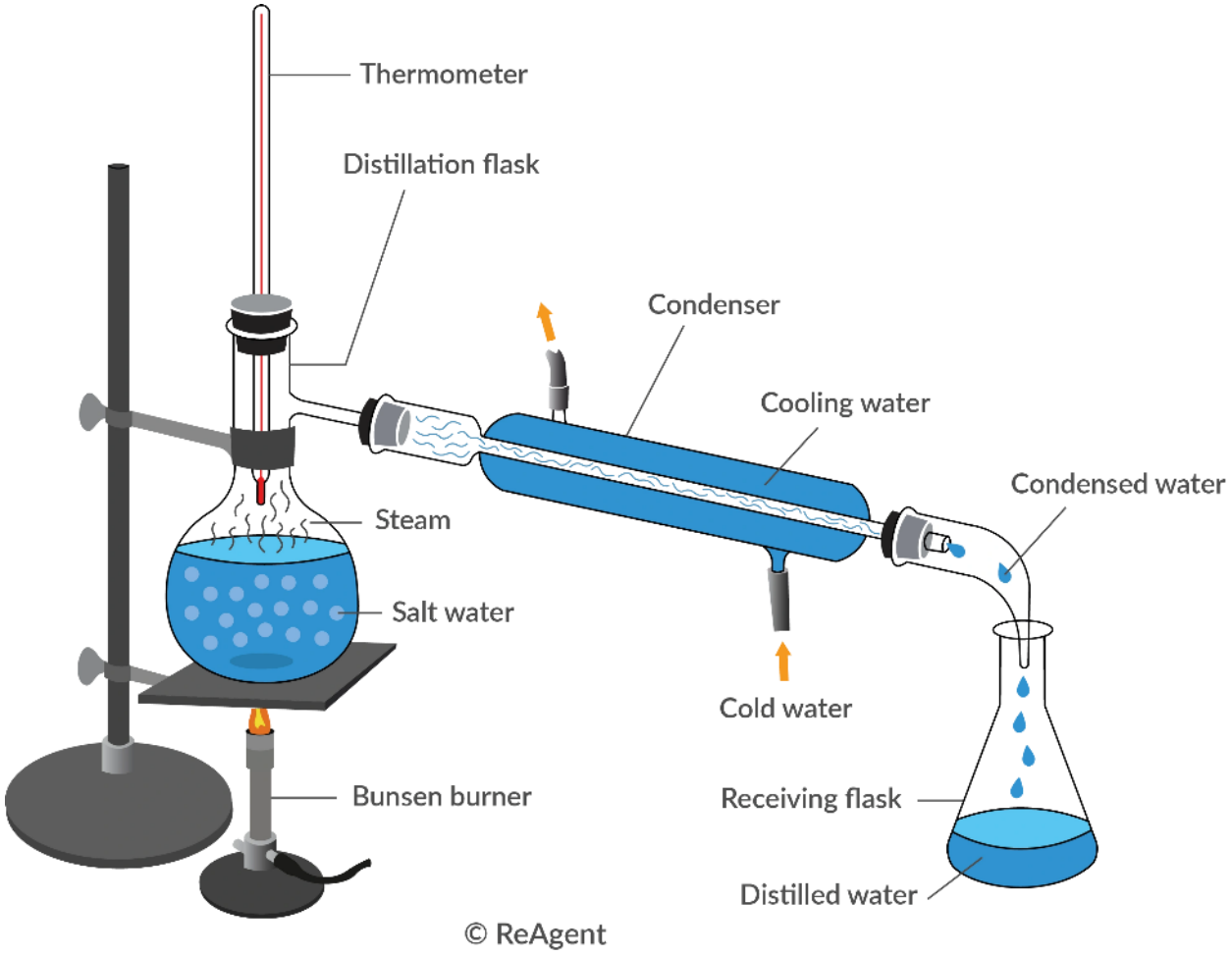


Figure 1: Throughput and accuracy on Imagenet of our methods compared to EfficientNets, trained on Imagenet1k only. The throughput is measured as the number of images processed per second on a V100 GPU. DeiT-B is identical to VIT-B, but the training is more adapted to a data-starving regime. It is learned in a few days on one machine. The symbol ⚗ refers to models trained with our transformer-specific distillation. See Table 5 for details and more models.

Distillation process

- **Distillation**
  - Process of purifying a substance, whereby pure substances are extracted form a mixture

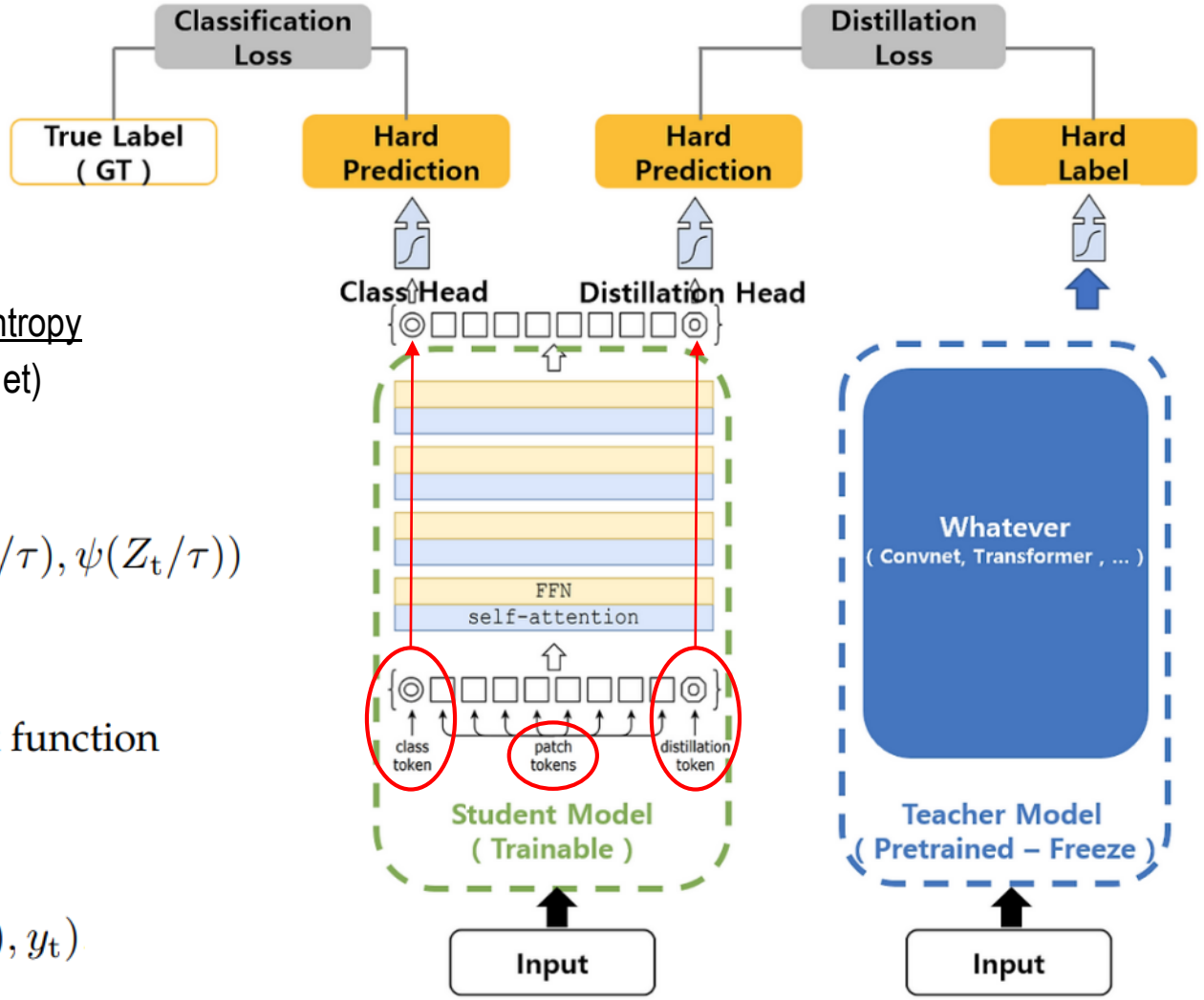


https://www.chemicals.co.uk/blog/what-is-the-distillation-process?srsltid=AfmBOooUR75W3I6F0kRHgtx3VJ1w1hvc8P9QpN6Kp_zdQAfSbJokMKjj

Architecture & Loss

Training data-efficient image transformers & distillation through attention, 2020

- **Distillation structure**
  - introduces two tokens:
    - Class Token → Standard supervised learning objective
    - Distillation Token → Teacher guidance learning
- **Output & Loss**
  - Class Token → Cross-entropy with GT labels
  - Distillation Token → KL-divergence with teacher outputs or Cross-entropy
    - The teacher : EfficientNet-style strong CNN model (e.g., RegNet)



Soft distillation

$$\mathcal{L}_{\text{global}} = (1-\lambda)\mathcal{L}_{\text{CE}}(\psi(Z_{\text{s}}), y) + \lambda\tau^2 \text{KL}(\psi(Z_{\text{s}}/\tau), \psi(Z_{\text{t}}/\tau))$$

Z_{s} the logits of the student model
Z_{t} be the logits of the teacher model
ground truth labels y, and ψ the softmax function

Hard distillation

$$\mathcal{L}_{\text{global}}^{\text{hardDistill}} = \frac{1}{2}\mathcal{L}_{\text{CE}}(\psi(Z_s), y) + \frac{1}{2}\mathcal{L}_{\text{CE}}(\psi(Z_s), y_{\text{t}})$$

https://hyoseok-personality.tistory.com/entry/Paper-Review-DeiT-Training-data-efficient-image-transformers-distillation-through-attention

Experiments

Table 2: We compare on ImageNet [42] the performance (top-1 acc., %) of the student as a function of the teacher model used for distillation.

Teacher Models	acc.	Student: DeiT-B⚗ pretrain	↑384
DeiT-B	81.8	81.9	83.1
RegNetY-4GF	80.0	82.7	83.6
RegNetY-8GF	81.7	82.7	83.8
RegNetY-12GF	82.4	83.1	84.1
RegNetY-16GF	82.9	83.1	84.2

Different teacher with inductive bias !

Clearer solution with answer sheet !

method ↓	Supervision label	Supervision teacher	ImageNet top-1 (%) Ti 224	S 224	B 224	B↑384
DeiT– no distillation	✓	✗	72.2	79.8	81.8	83.1
DeiT– usual distillation	✗	soft	72.2	79.8	81.8	83.2
DeiT– hard distillation	✗	hard	74.3	80.9	83.0	84.0
DeiT⚗: class embedding	✓	hard	73.9	80.9	83.0	84.2
DeiT⚗: distil. embedding	✓	hard	74.6	81.1	83.1	84.4
DeiT⚗: class+distillation	✓	hard	74.5	81.2	83.4	84.5

Table 3: Distillation experiments on Imagenet with DeiT, 300 epochs of pre-training. We report the results for our new distillation method in the last three rows. We separately report the performance when classifying with only one of the class or distillation embeddings, and then with a classifier taking both of them as input. In the last row (class+distillation), the result correspond to the late fusion of the class and distillation classifiers.

Summary

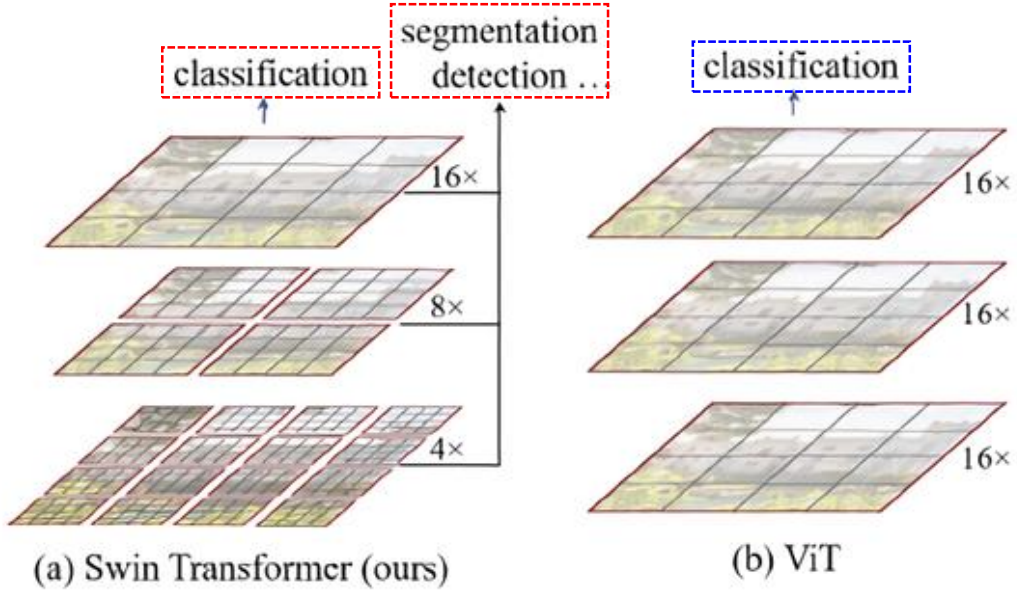
- **Key contributions**

Contribution	Description
Knowledge Distillation	Transfer inductive bias from CNN teacher
Distillation Token	A new token that gathers teacher's guidance
Pure Transformer Architecture	No convolutional hybrid structures
Optimized Training Recipe	Strong augmentation + regularization + optimization strategy

"DeiT proves that Transformers can be trained efficiently even without massive datasets."

[Swin Transformer]

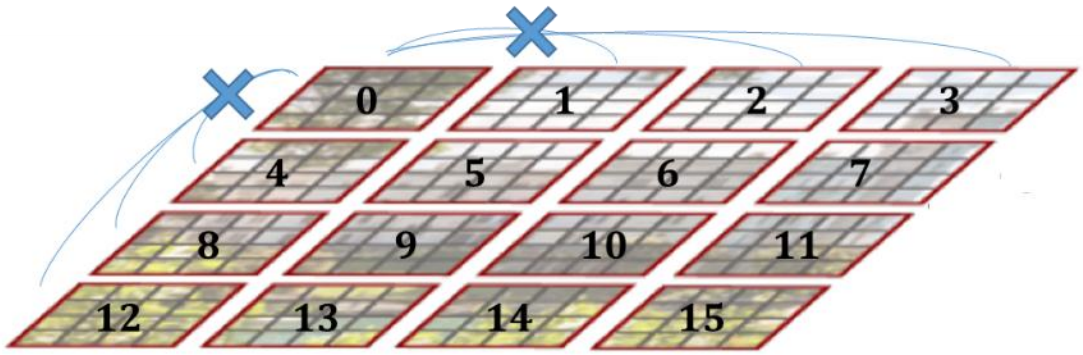
- **Motivation**
  - Vision Transformer (ViT) struggles with:
    - Quadratic complexity of global self-attention → proportional to the square of (# of patches)
    - Lack of hierarchical representation (unlike CNNs)
      - Limited robustness to scale variations and dense prediction tasks such as detections and segmentations
- **Goal** of the Swin (i.e., shifted window) transformer
  - The combination of
    - Global modeling power of Transformers
    - Efficient, hierarchical spatial inductive bias of CNNs



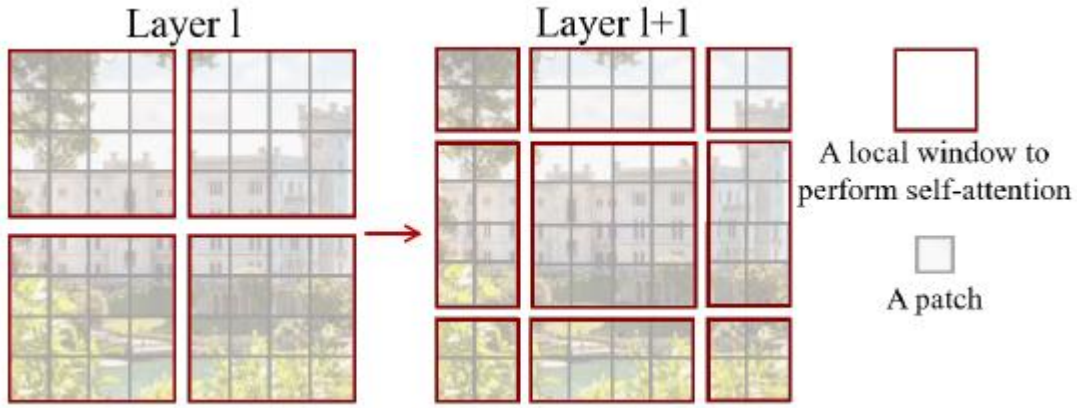
Window mechanism

- **Shifted window mechanism**

Operation	Description	Result
W-MSA	Self-attention inside non-overlapping fixed windows	Efficient local self-attention
SW-MSA	Windows shift by (Window size / 2) → attention across previous window boundaries	Enables cross-window information exchange



Window-based self attention

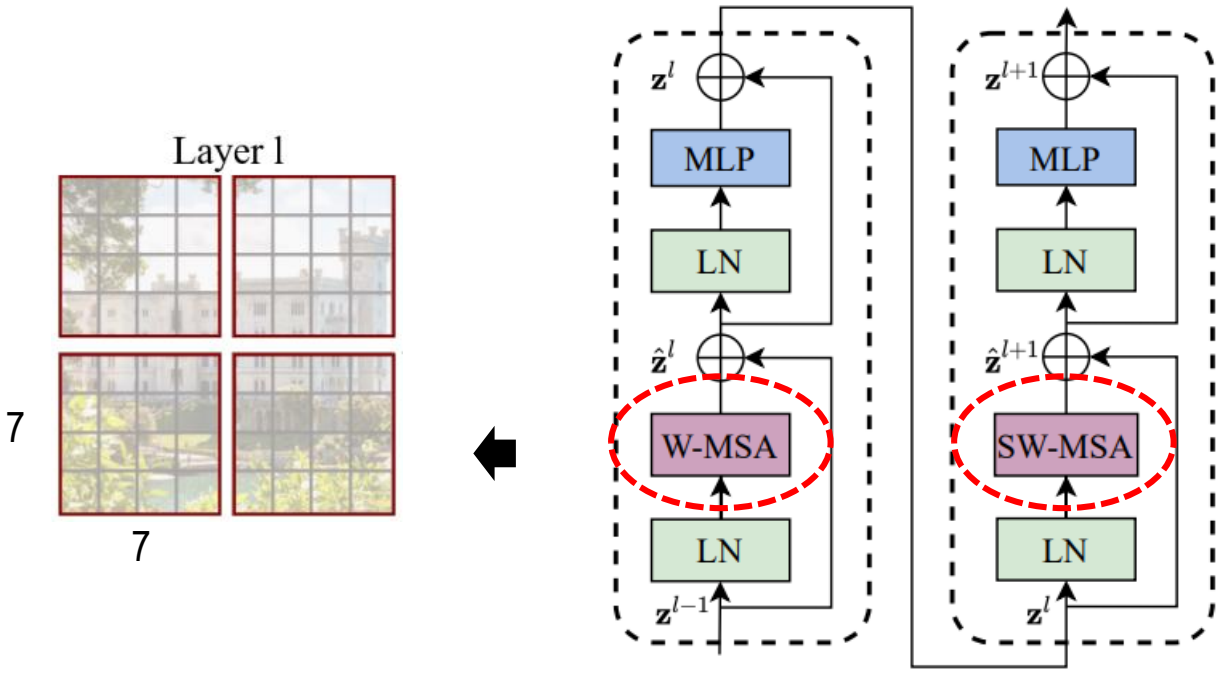


Window-MSA **Shifted Window-MSA**

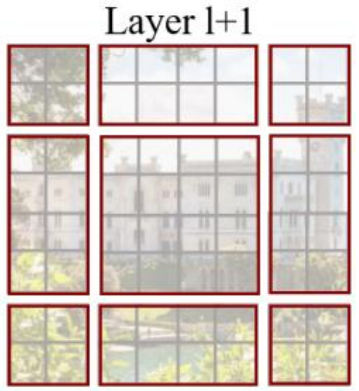
https://lcyking.tistory.com/entry/%EB%85%BC%EB%AC%B8%EB%A6%AC%EB%B7%B0-Swin-Transformer-Hierarchical-Vision-Transformer-using-Shifted-Windows

Architecture (1/2)

- **Two successive Swin Transformer block**



- Shift windows (M=7) by M/2 patches
  (e.g., 7 → shift by 3)



- Cyclic shift, then compute attention
  - Same window
  - Different window : masked out

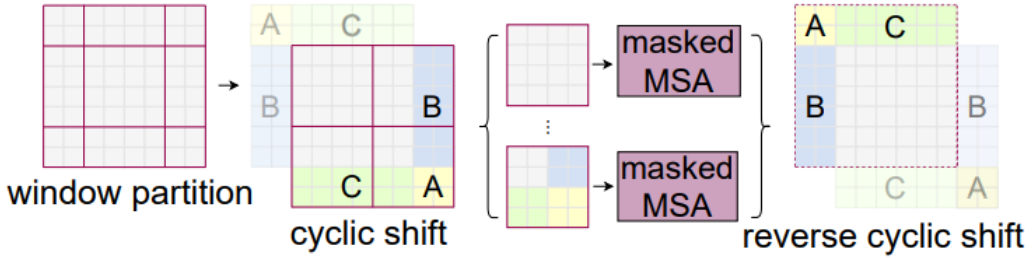
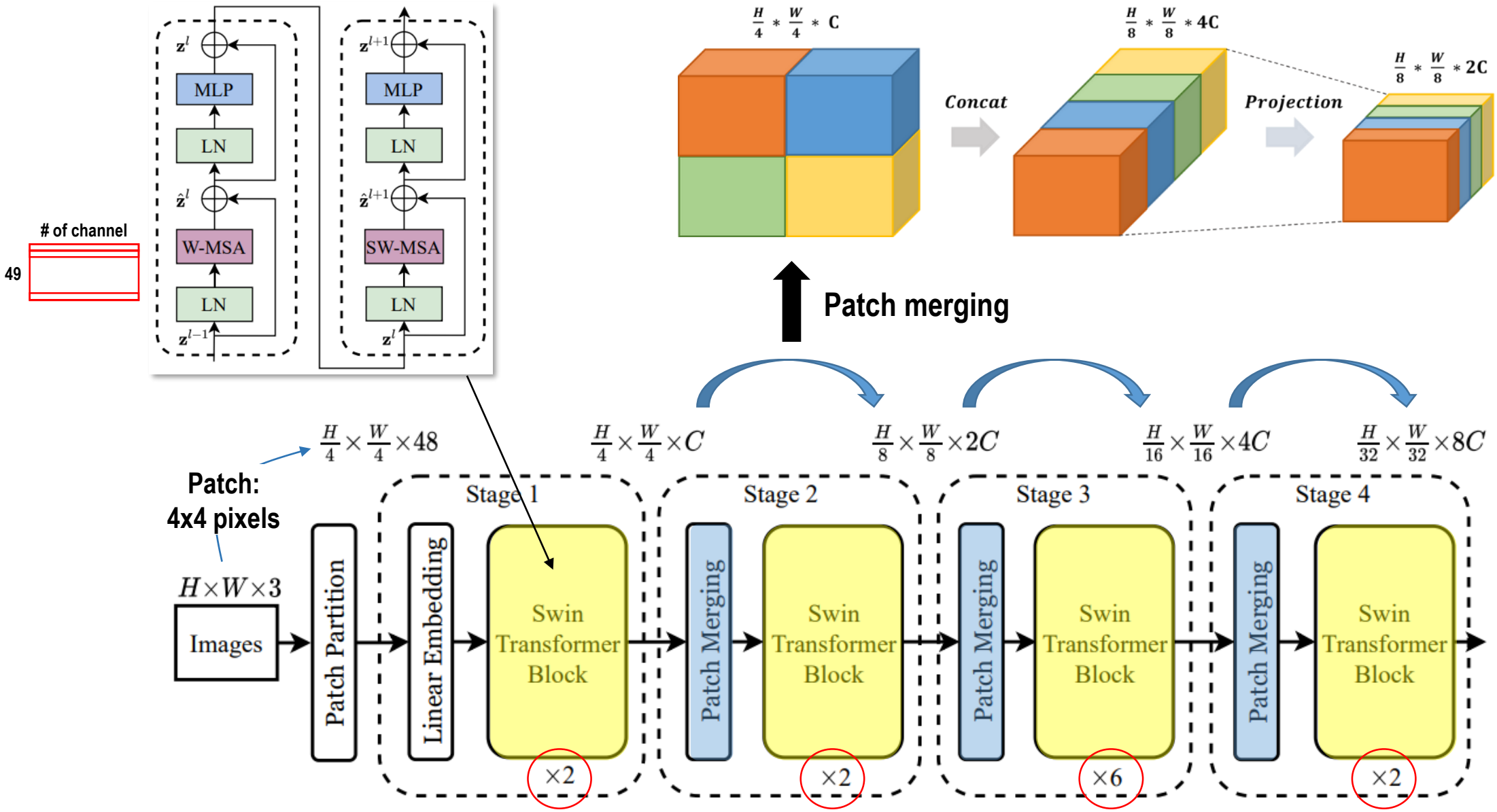


Figure 4. Illustration of an efficient batch computation approach for self-attention in shifted window partitioning.

Architecture (2/2)



https://lcyking.tistory.com/entry/%EB%85%BC%EB%AC%B8%EB%A6%AC%EB%B7%B0-Swin-Transformer-Hierarchical-Vision-Transformer-using-Shifted-Windows

Complexity

- ViT vs. Swin in terms of the computational complexity

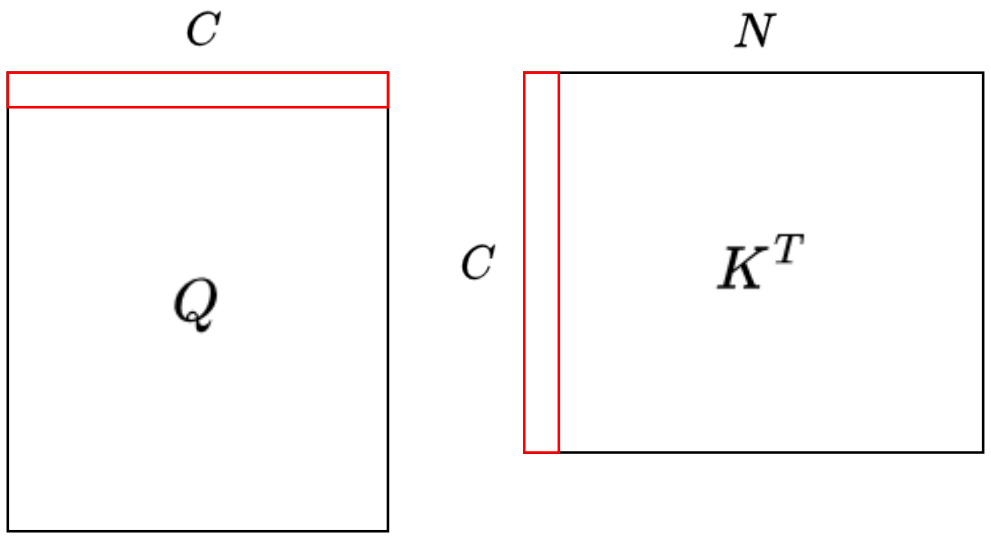
Attention Type	Complexity	Practical Impact
Global MSA (ViT)	$4HWC^2 + 2(HW)^2C \Rightarrow O\left((HW)^2\right)$	Quadratic w.r.t. the number of patches
Window MSA (Swin)	$4HWC^2 + 2M^2HWC \Rightarrow O(HW)$	Linear w.r.t. the number of patches M^2 indicates the number of patches in a window

$$2M^2HWC = \frac{HW}{M^2} 2\left((M^2)^2C\right)$$

Computations in ViT

$N = HW$

(# of patches)



$$\Rightarrow \quad N \cdot 2C \cdot N \quad \Rightarrow \quad O\left((HW)^2\right)$$

Experiments

(a) Regular ImageNet-1K trained models

method	image size	#param.	FLOPs	throughput (image / s)	ImageNet top-1 acc.
RegNetY-4G [48]	224^2	21M	4.0G	1156.7	80.0
RegNetY-8G [48]	224^2	39M	8.0G	591.6	81.7
RegNetY-16G [48]	224^2	84M	16.0G	334.7	82.9
EffNet-B3 [58]	300^2	12M	1.8G	732.1	81.6
EffNet-B4 [58]	380^2	19M	4.2G	349.4	82.9
EffNet-B5 [58]	456^2	30M	9.9G	169.1	83.6
EffNet-B6 [58]	528^2	43M	19.0G	96.9	84.0
EffNet-B7 [58]	600^2	66M	37.0G	55.1	84.3
ViT-B/16 [20]	384^2	86M	55.4G	85.9	77.9
ViT-L/16 [20]	384^2	307M	190.7G	27.3	76.5
DeiT-S [63]	224^2	22M	4.6G	940.4	79.8
DeiT-B [63]	224^2	86M	17.5G	292.3	81.8
DeiT-B [63]	384^2	86M	55.4G	85.9	83.1
Swin-T	224^2	29M	4.5G	755.2	81.3
Swin-S	224^2	50M	8.7G	436.9	83.0
Swin-B	224^2	88M	15.4G	278.1	83.5
Swin-B	384^2	88M	47.0G	84.7	84.5

(b) ImageNet-22K pre-trained models

method	image size	#param.	FLOPs	throughput (image / s)	ImageNet top-1 acc.
R-101x3 [38]	384^2	388M	204.6G	-	84.4
R-152x4 [38]	480^2	937M	840.5G	-	85.4
ViT-B/16 [20]	384^2	86M	55.4G	85.9	84.0
ViT-L/16 [20]	384^2	307M	190.7G	27.3	85.2
Swin-B	224^2	88M	15.4G	278.1	85.2
Swin-B	384^2	88M	47.0G	84.7	86.4
Swin-L	384^2	197M	103.9G	42.1	87.3

Table 1. Comparison of different backbones on ImageNet-1K classification. Throughput is measured using the GitHub repository of [68] and a V100 GPU, following [63].

Summary

- **Architectural benefits**

	Purpose	Benefit
Hierarchical Patch Representation	Stage-wise downsampling & channel increase	Works well for detection/segmentation
Shifted Window-based Self-Attention (W-MSA / SW-MSA)	Local attention with periodic cross-window communication	Linear complexity w.r.t. # of patches
Patch Merging	Spatial dimension ↓, feature dimension ↑	Builds feature pyramids
Cyclic Shift + Attention Masking	Bridges windows while preventing invalid attention	Preserves locality & global context

- **Comparison between ViT and Swin Transformer**

	ViT	Swin Transformer
Structure	Flat tokens	Hierarchical stages
Attention	Global	Window-based + shifted
Complexity	$O((HW)^2)$	$O(HW)$
Best suited for	Classification	Detection/Segmentation

Parameter count

- **ViT**

	Hidden Dim	Heads	Depth	Params
ViT-Tiny (Ti/16)	192	3	12	5.7M
ViT-Small (S/16)	384	6	12	22M
ViT-Base (B/16)	768	12	12	86M
ViT-Large (L/16)	1024	16	24	307M
ViT-Huge (H/14)	1280	16	32	632M

- **DeiT**

	Params
DeiT-Tiny	5M
DeiT-Small	22M
DeiT-Base	86M
DeiT-Base + Distilled	87M

- **Swin Transformer**

	Params
Swin-Tiny (Swin-T)	28M
Swin-Small (Swin-S)	50M
Swin-Base (Swin-B)	88M
Swin-Large (Swin-L)	197M

[CLIP]

- **CLIP** (**C**ontrastive **L**anguage-**I**mage **P**re-training)
- **Connecting Text and Images**



Overview

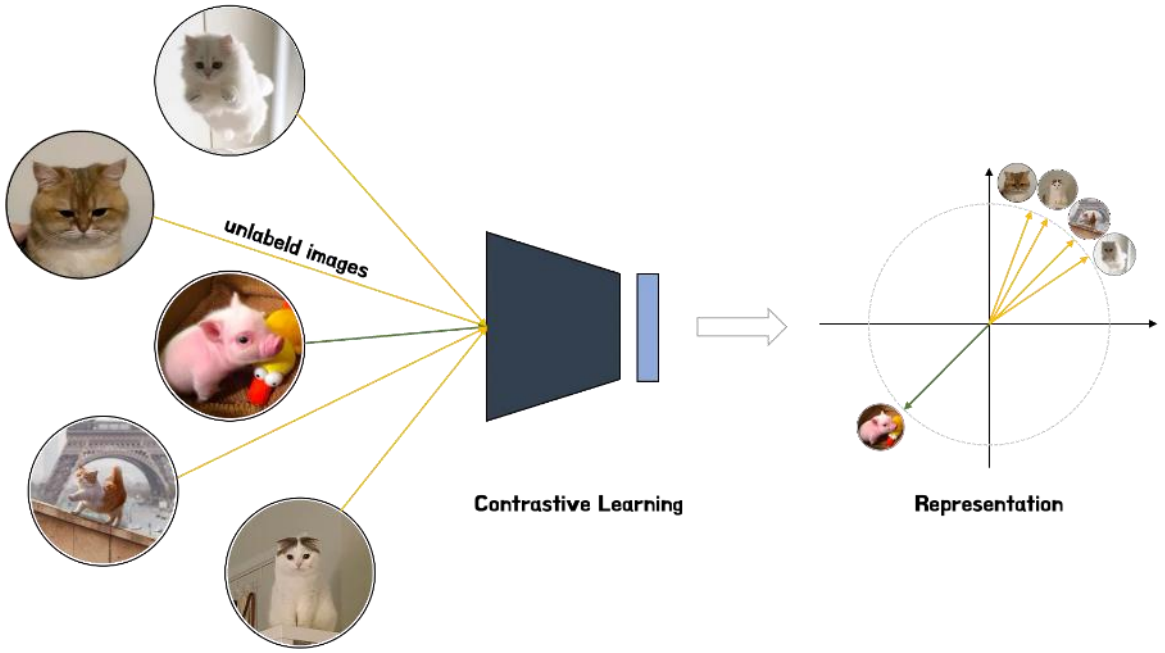
- **Motivation**
  - Traditional vision models rely heavily on manually labeled datasets
  - Hard to scale because labeling is expensive and limited in vocabulary
  - Models lack zero-shot capability
    → Need retraining for every new task

- **Key Insight**
  - Billions of images with natural language descriptions on the internet
    → Text can serve as universal supervision for visual concepts

- **Goal**
  - Learn visual concepts directly from natural language

- **Main Idea**
  - Contrastive learning

Contrastive learning



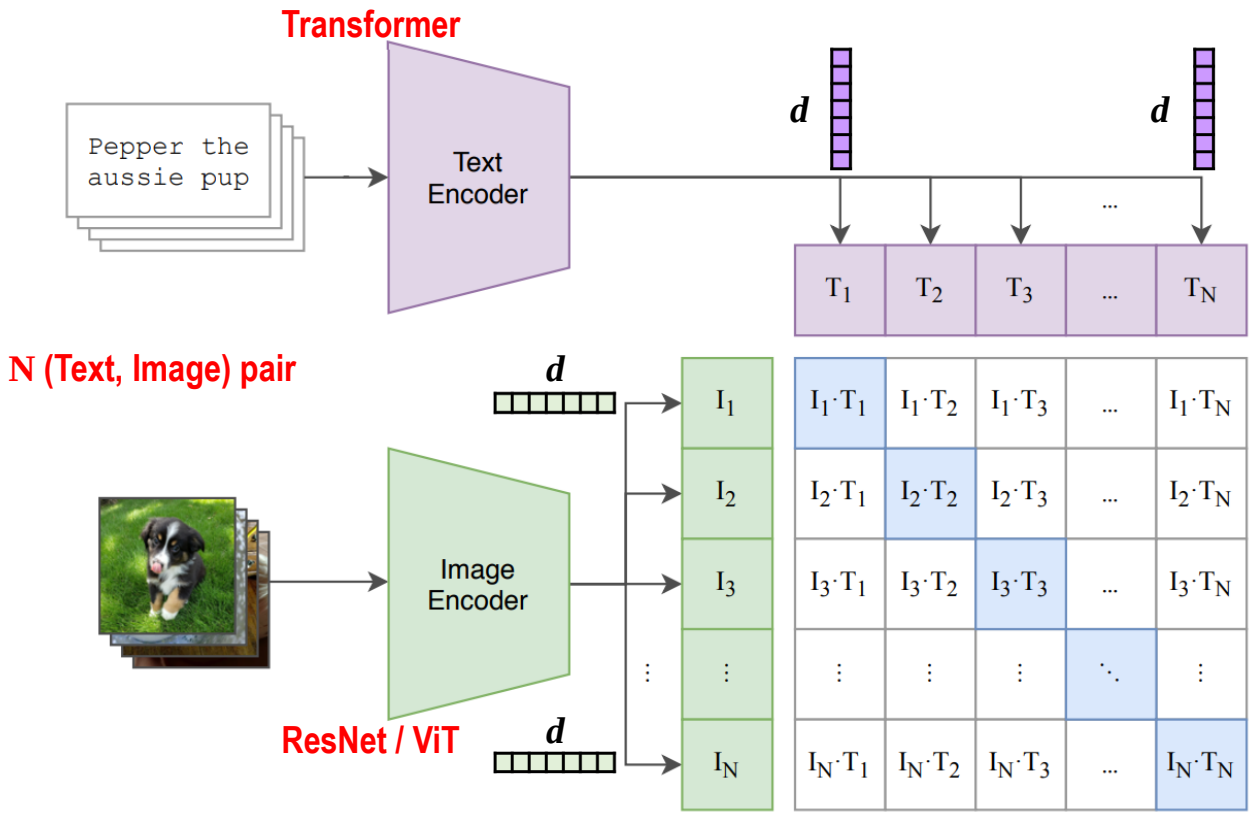
https://www.blossominkyung.com/deeplearning/contrastive-learning

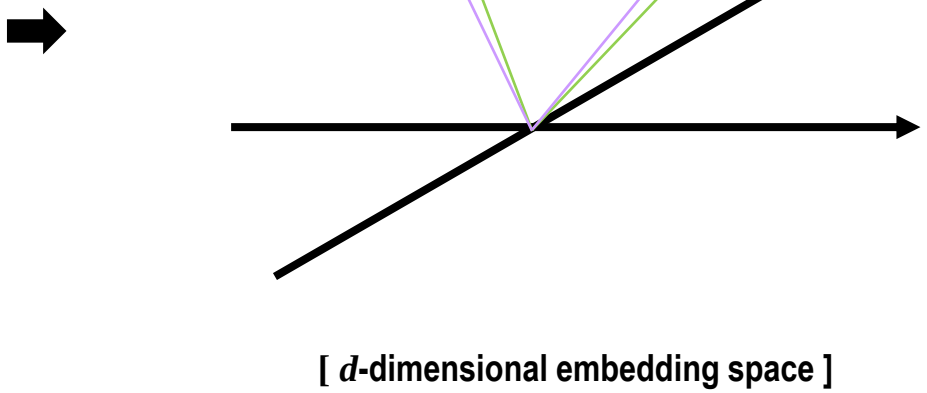
New dataset

- **Dataset**
  - Popular existing datasets
    - MS-COCO, Visual Genome : Good quality but small (About 100K)
    - YFCC100M : Varying quality and large (About 100M)

  - New 400M (Image, Text) pair dataset → More robust generalization → Zero-shot
    - Collected from publicly available internet sources
    - No manual labeling required
    - Wide coverage of real-world concepts

Architecture

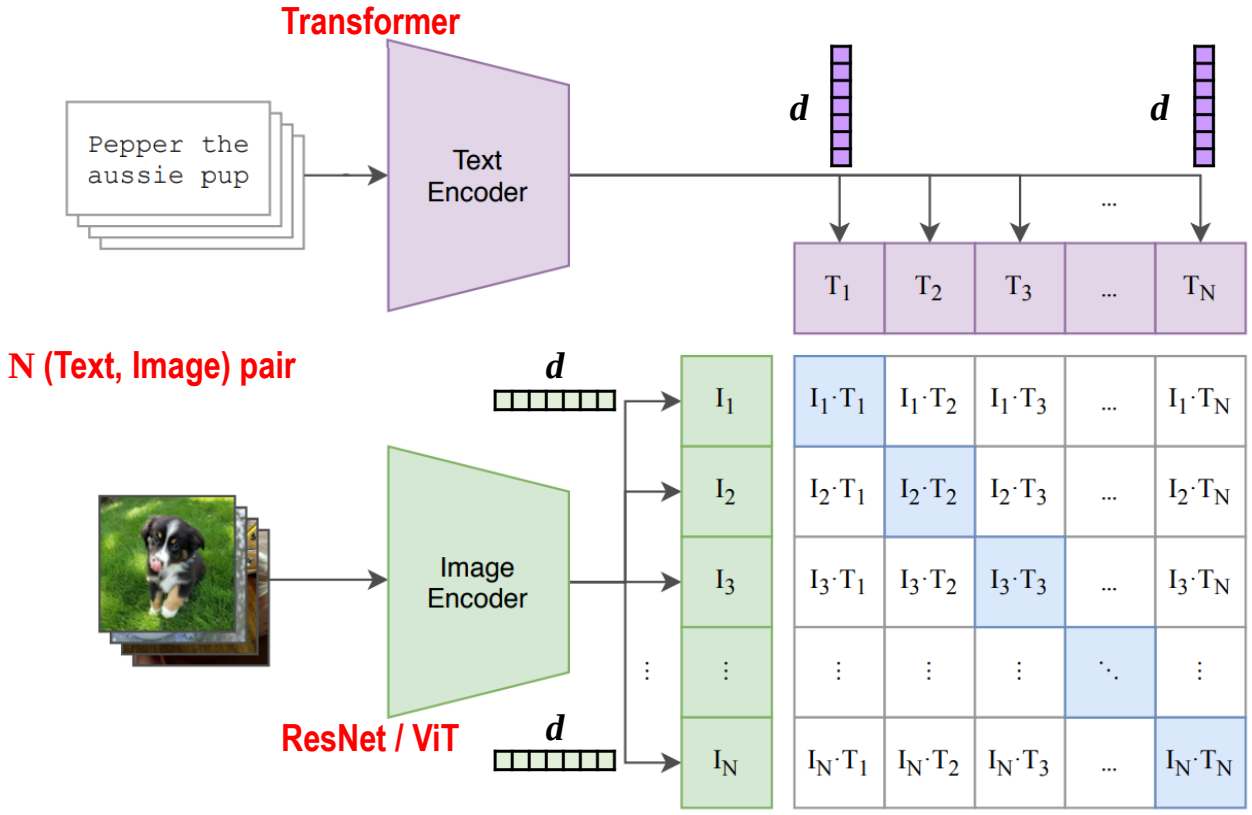




[*d*-dimensional embedding space]

Training (1/2)

- **Contrastive learning**



- **Mini-batch size** : 32,768
- The largest ResNet model (RN50X64) : **18 days train on 592 V100 GPUs**
- ViT-L/14 performs best

```
# image_encoder - ResNet or Vision Transformer
# text_encoder  - CBOW or Text Transformer
# I[n, h, w, c] - minibatch of aligned images
# T[n, l]       - minibatch of aligned texts
# W_i[d_i, d_e] - learned proj of image to embed
# W_t[d_t, d_e] - learned proj of text to embed
# t             - learned temperature parameter

# extract feature representations of each modality
I_f = image_encoder(I) #[n, d_i]
T_f = text_encoder(T)  #[n, d_t]

# joint multimodal embedding [n, d_e]
I_e = l2_normalize(np.dot(I_f, W_i), axis=1)
T_e = l2_normalize(np.dot(T_f, W_t), axis=1)

# scaled pairwise cosine similarities [n, n]
logits = np.dot(I_e, T_e.T) * np.exp(t)

# symmetric loss function
labels = np.arange(n)
loss_i = cross_entropy_loss(logits, labels, axis=0)
loss_t = cross_entropy_loss(logits, labels, axis=1)
loss   = (loss_i + loss_t)/2
```

Figure 3. Numpy-like pseudocode for the core of an implementation of CLIP.

Training (2/2)

- **Contrastive loss (i.e., InfoNCE)**

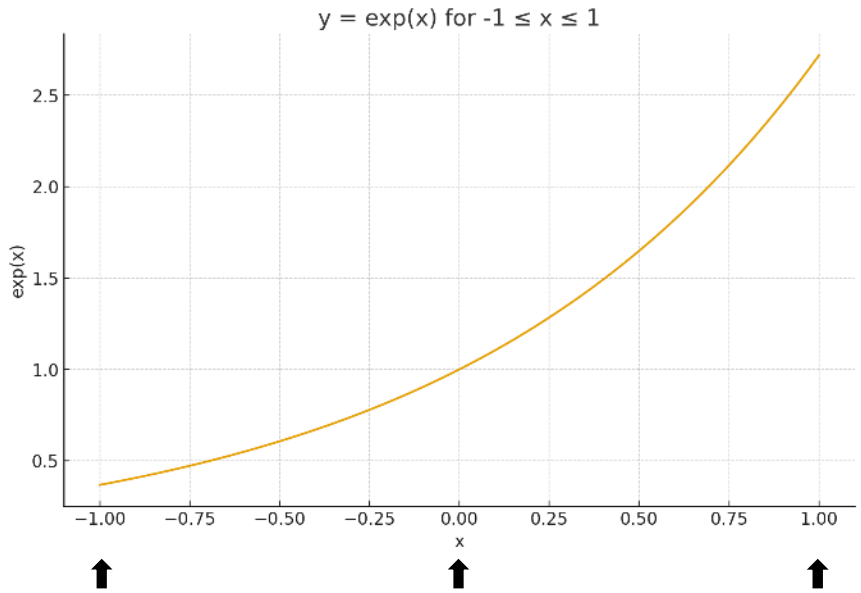
- sim: cosine similarity
- τ: temperature

$$\mathcal{L} = -\frac{1}{N}\sum_{i=1}^{N}\log\frac{\exp(\text{sim}(z^i_{\text{img}}, z^i_{\text{text}})/\tau)}{\sum_{j=1}^{N}\exp(\text{sim}(z^i_{\text{img}}, z^j_{\text{text}})/\tau)}$$

High similarity

$I_1 \cdot T_1$	$I_1 \cdot T_2$	$I_1 \cdot T_3$	...	$I_1 \cdot T_N$
$I_2 \cdot T_1$	$I_2 \cdot T_2$	$I_2 \cdot T_3$	...	$I_2 \cdot T_N$
$I_3 \cdot T_1$	$I_3 \cdot T_2$	$I_3 \cdot T_3$	...	$I_3 \cdot T_N$
⋮	⋮	⋮	⋱	⋮
$I_N \cdot T_1$	$I_N \cdot T_2$	$I_N \cdot T_3$	...	$I_N \cdot T_N$

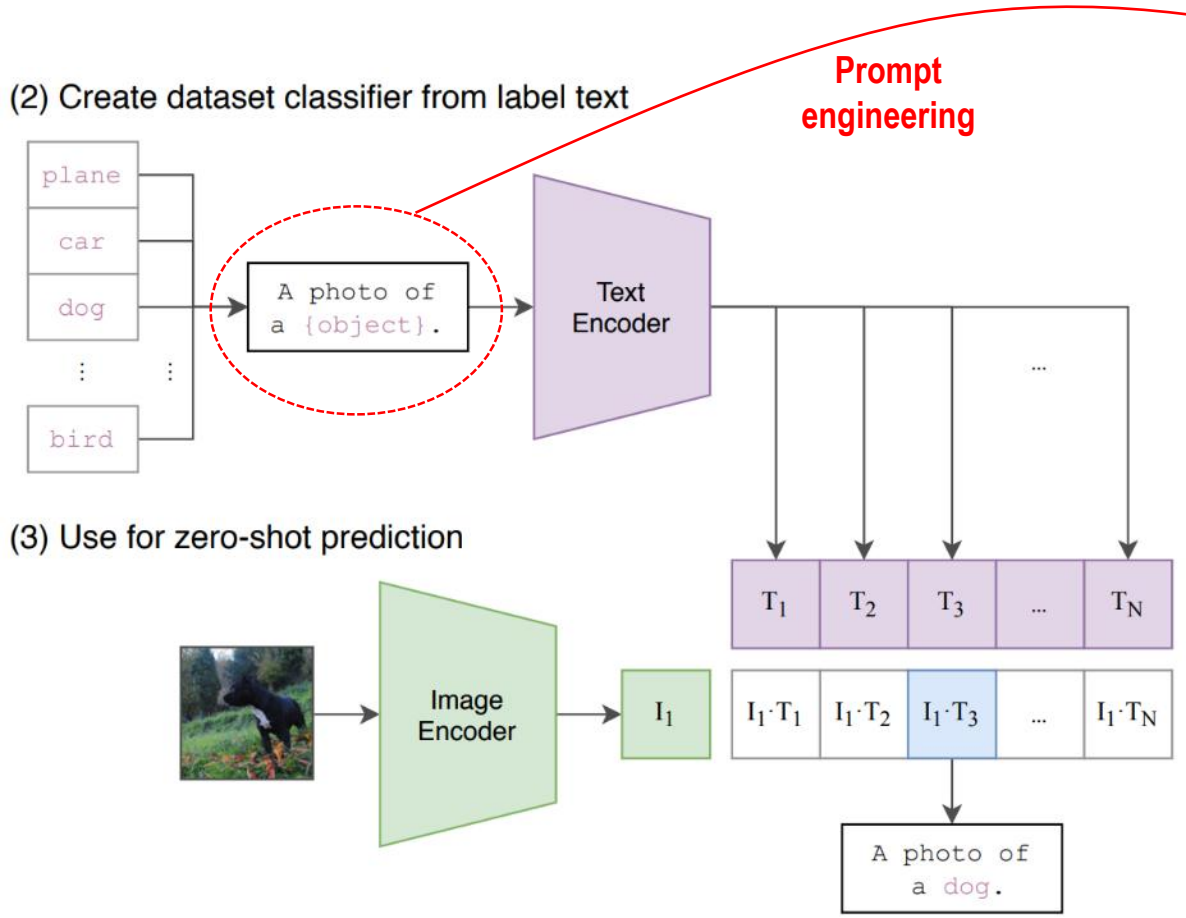
For each row



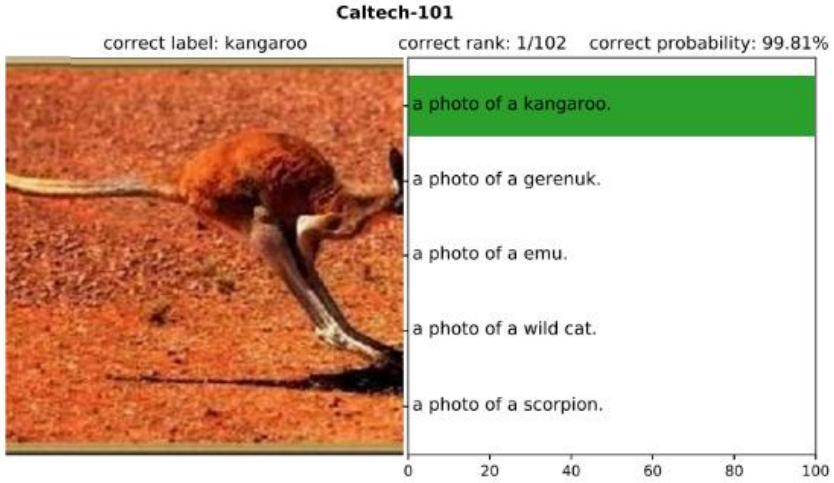
Opposite direction | Orthogonal | Same direction

Inference (1/2)

- **Zero-shot linear classifier**
  - embedding the names or descriptions of the target dataset's classes



- Aligns with natural caption style found in the web data CLIP was trained on
- Reduces semantic ambiguity of standalone class names (e.g., apple = fruit vs. brand)
- Improves zero-shot accuracy compared to using the class name alone
  → Increased accuracy on ImageNet by 1.3%
- Creates consistent structure for all classes
  → stable embeddings



Inference (2/2)

- **For fine-grained image classification datasets**
  - Oxford-IIIT Pets, using “A photo of a {label}, a type of pet.”
  - Food101, “a type of food“
  - FGVC, “a type of aircraft “
  - For OCR datasets, putting quotes around the text or number

- **Ensembling over the embedding space**
  - ‘A photo of a big {label}” and “A photo of a small {label}”

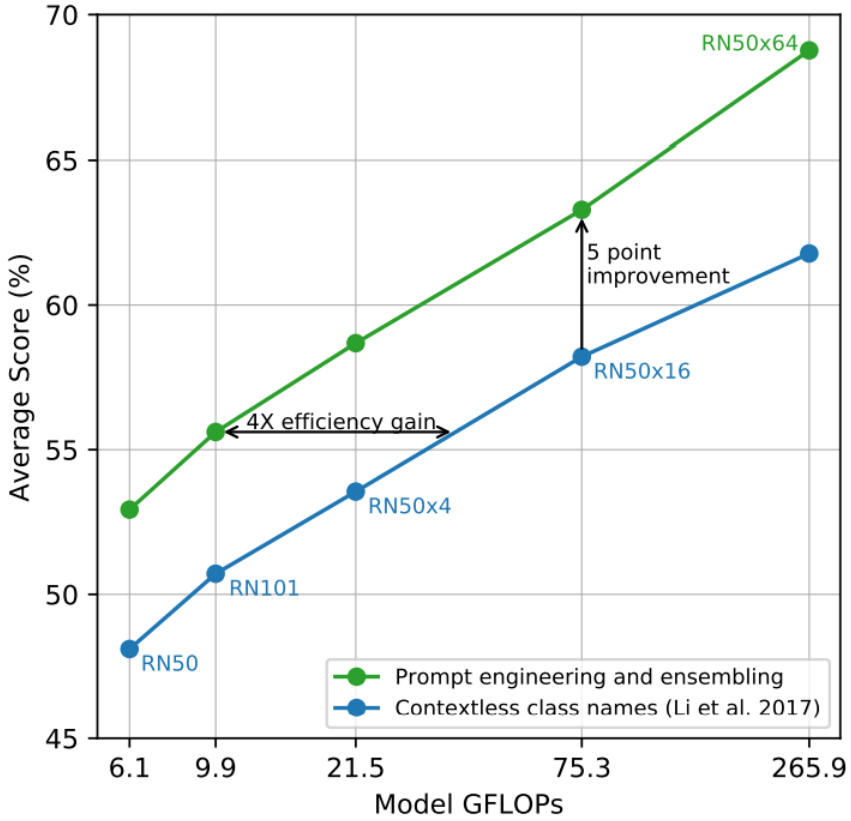


Figure 4. **Prompt engineering and ensembling improve zero-shot performance.** Compared to the baseline of using contextless class names, prompt engineering and ensembling boost zero-shot classification performance by almost 5 points on average across 36 datasets. This improvement is similar to the gain from using 4 times more compute with the baseline zero-shot method but is “free” when amortized over many predictions.

Zero-shot experiments (1/3)

- **Comparison with the supervised model**
  - Better performance on action recognition in videos (①,②)
    → Natural language provides wider supervision for visual concepts involving verbs
  - Bad performance on specialized, complex, or abstract tasks (③,④,⑤,⑥)
    - e.g., satellite, medical, synthetic image sets

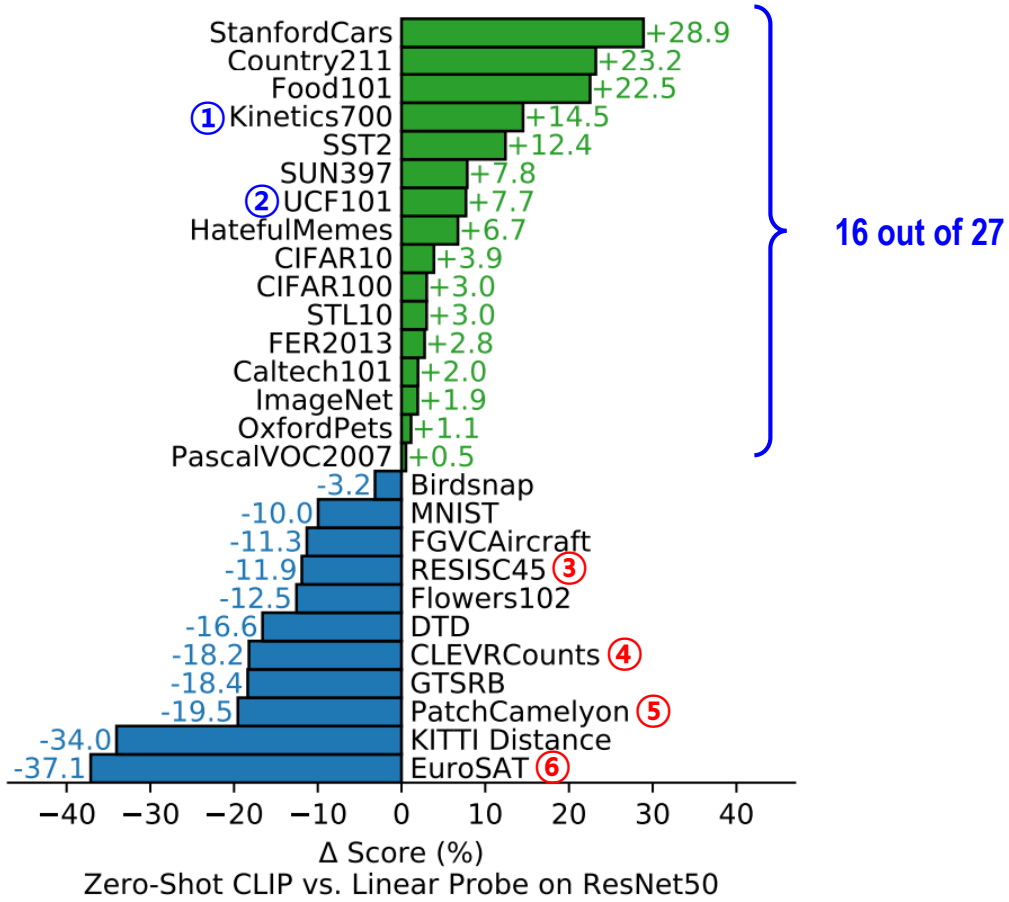


Figure 5. **Zero-shot CLIP is competitive with a fully supervised baseline.** Across a 27 dataset eval suite, a zero-shot CLIP classifier outperforms a fully supervised linear classifier fitted on ResNet-50 features on 16 datasets, including ImageNet.

Zero-shot experiments (2/3)

- **Comparison with few-shot linear probes**
  - Zero-shot CLIP performs as well as a 4-shot linear probe
    - Linear-probe CLIP setup,
      - Image encoder of CLIP (e.g. a ResNet-50) is kept frozen
      - On top of the frozen image encoder,
        a simple linear classifier (linear layer / logistic regression) is added
  - Zero-shot CLIP even surpasses few-shot linear classifiers
  - Few-shot linear probes may overfit
    when only 1–2 labeled samples per class are available
  - CLIP provides strong label efficiency and broad generalization
    without any task-specific training

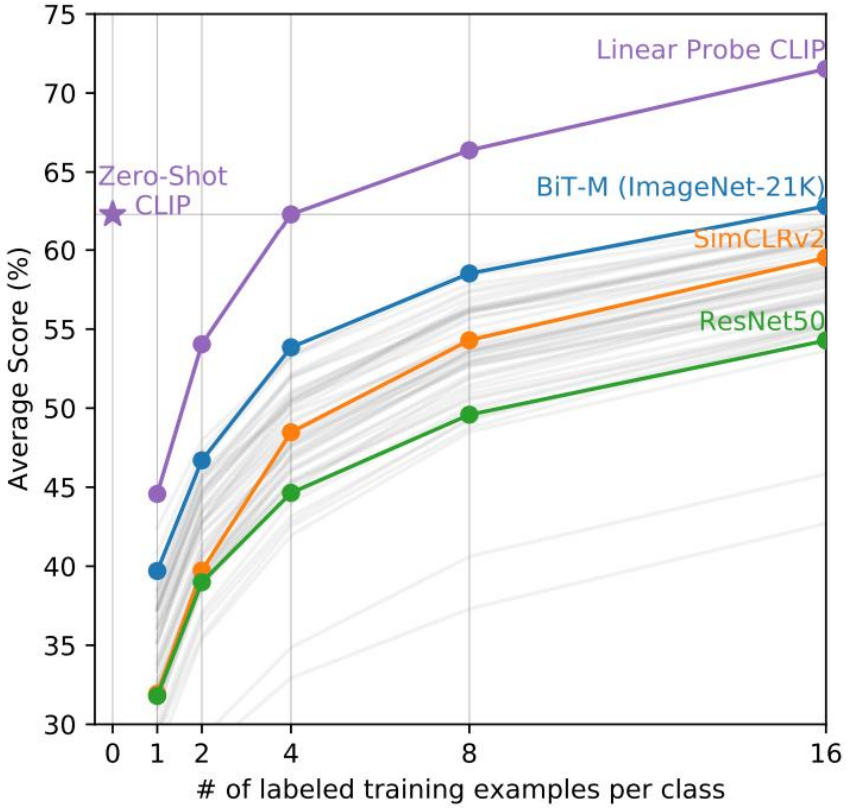
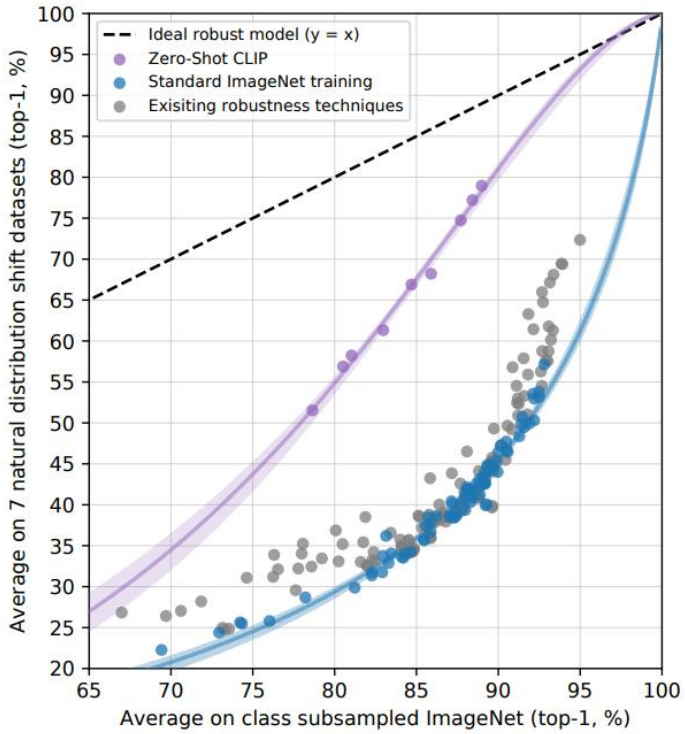


Figure 6. **Zero-shot CLIP outperforms few-shot linear probes.** Zero-shot CLIP matches the average performance of a 4-shot linear classifier trained on the same feature space and nearly matches the best results of a 16-shot linear classifier across publicly available models. For both BiT-M and SimCLRv2, the best performing model is highlighted. Light gray lines are other models in the eval suite. The 20 datasets with at least 16 examples per class were used in this analysis.

Zero-shot experiments (3/3)

- **Distribution shift of dataset**
  - CLIP is more robust to real-world distribution shift than supervised ImageNet models.



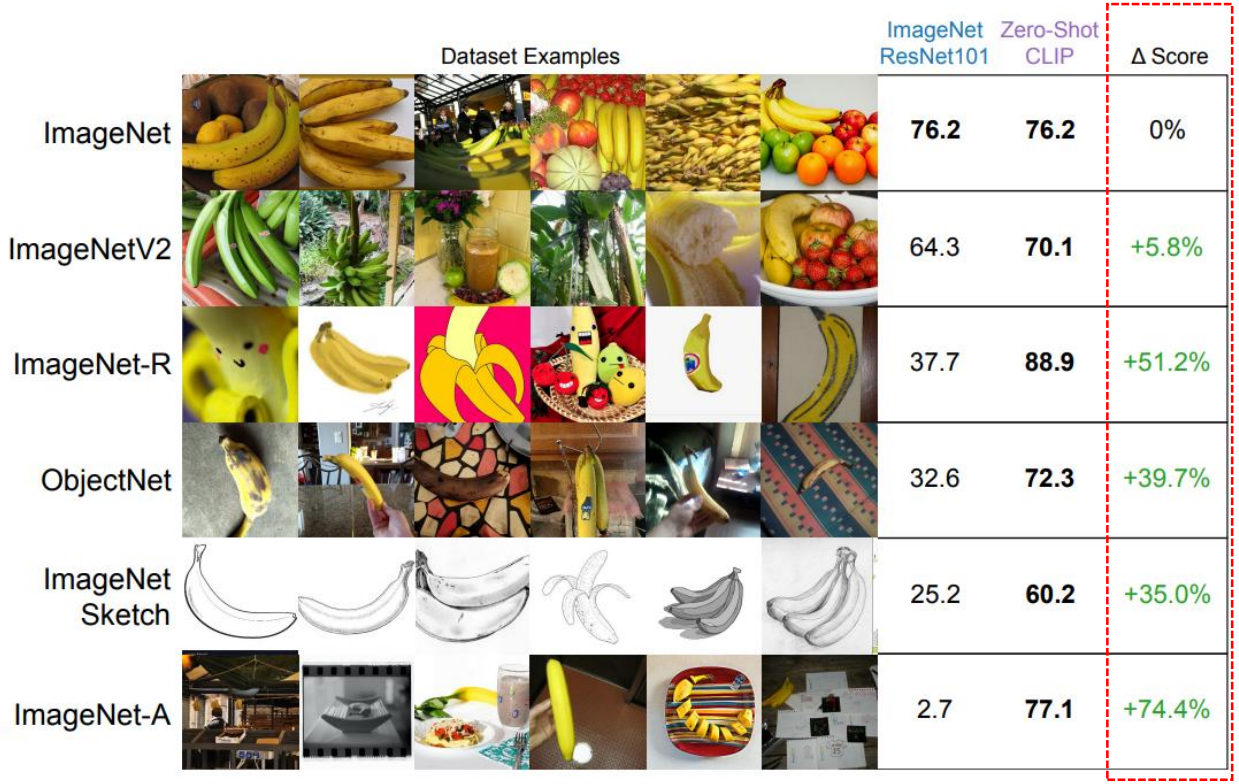


Figure 13. **Zero-shot CLIP is much more robust to distribution shift than standard ImageNet models.** (Left) An ideal robust model (dashed line) performs equally well on the ImageNet distribution and on other natural image distributions. Zero-shot CLIP models shrink this "robustness gap" by up to 75%. Linear fits on logit transformed values are shown with bootstrap estimated 95% confidence intervals. (Right) Visualizing distribution shift for bananas, a class shared across 5 of the 7 natural distribution shift datasets. The performance of the best zero-shot CLIP model, ViT-L/14@336px, is compared with a model that has the same performance on the ImageNet validation set, ResNet-101.

Summary

- **CLIP** shows that vision models trained on diverse web data **can act as general-purpose visual learners**
  - Learns directly from natural language supervision
  - Enables zero-shot generalization to unseen tasks
  - Bridges the gap between vision and language understanding

- **Strong robustness to distribution shift**
  - Reduces the need for expensive labeled datasets

A shift **from task-specific** training
to universally transferable representations !

[DINO]

- **Motivation**
  - Why self-supervised learning for vision?
    - Supervised learning requires massive labeled datasets — human labeling is expensive, slow, domain-dependent
    - Self-supervised learning (SSL) promises label-free pretraining
    - Vision Transformers (ViT) show global receptive field, which is ideal for self-supervision — but how to train them without labels?
- **Goal**
  - Enabling ViT to learn object-centric representations without any labels

DinoV2_Demo

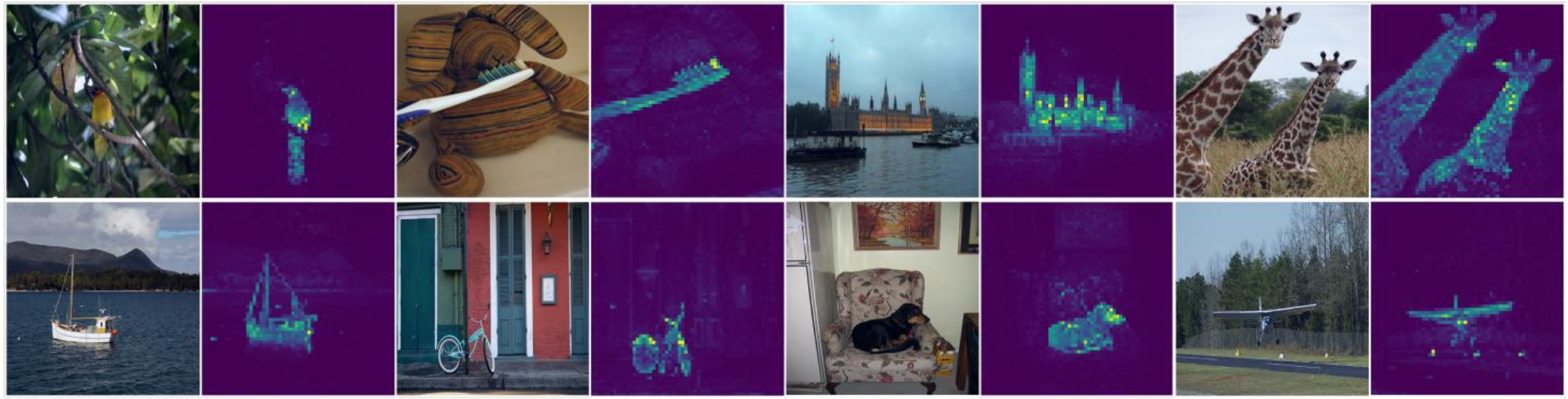
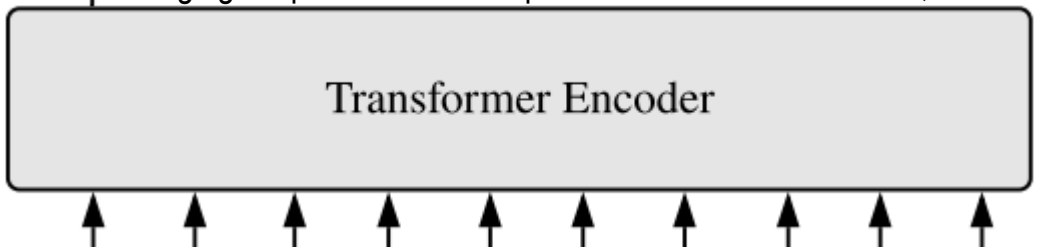


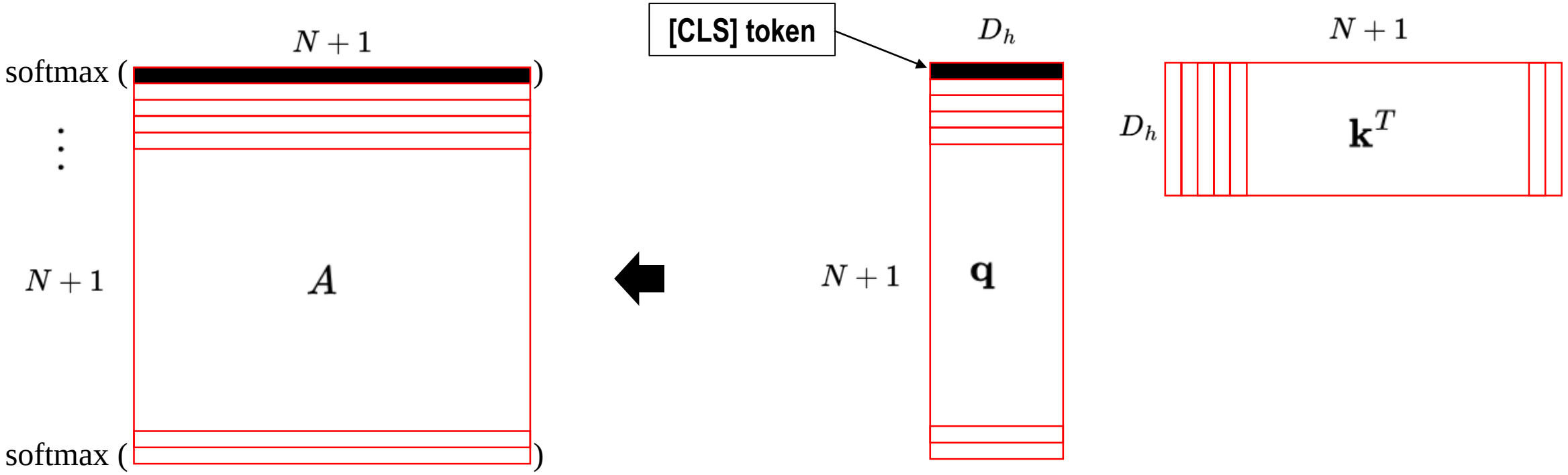
Figure 1: **Self-attention from a Vision Transformer with 8×8 patches trained with no supervision.** We look at the self-attention of the `[CLS]` token on the heads of the last layer. This token is not attached to any label nor supervision. These maps show that the model automatically learns class-specific features leading to unsupervised object segmentations.

(Recap.) Self-attention map

Emerging Properties in Self-Supervised Vision Transformers, 2021



$$A = \text{softmax}\left(\mathbf{q}\mathbf{k}^\top / \sqrt{D_h}\right)$$



Architecture (1/6)

- **Student–Teacher framework** (Self-distillation with no labels)
  - A student and a teacher with identical architectures
  - Teacher parameters updated via EMA (Exponential Moving Average) of the student

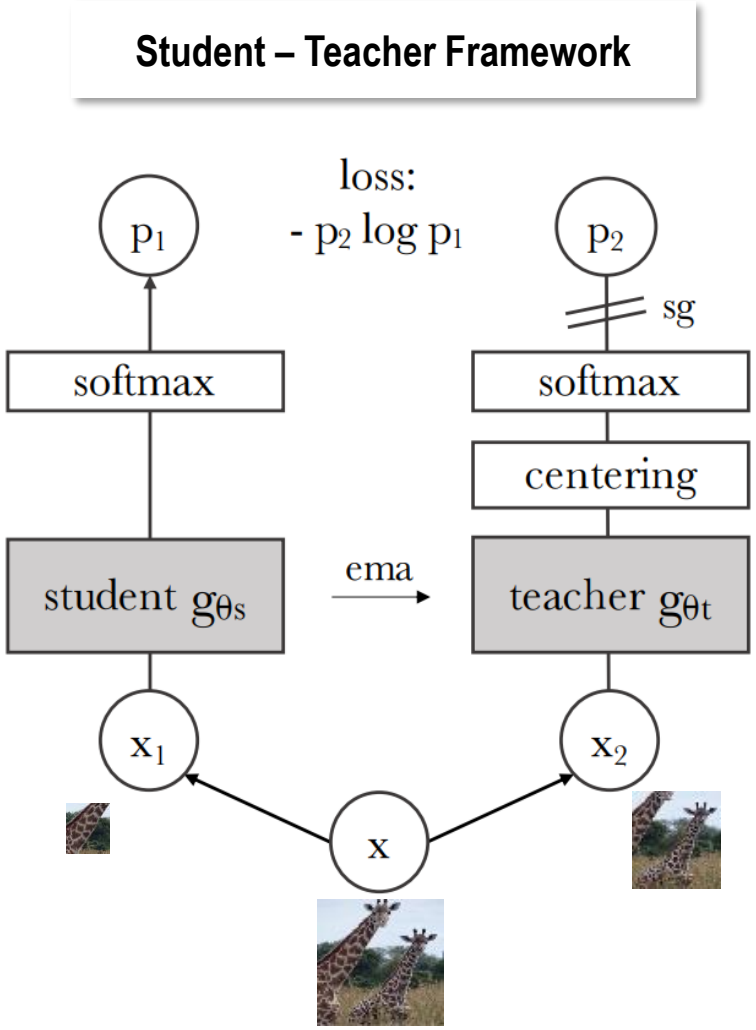
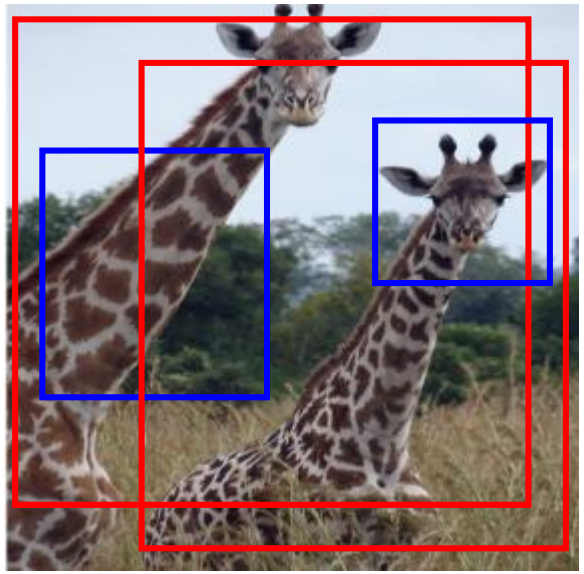


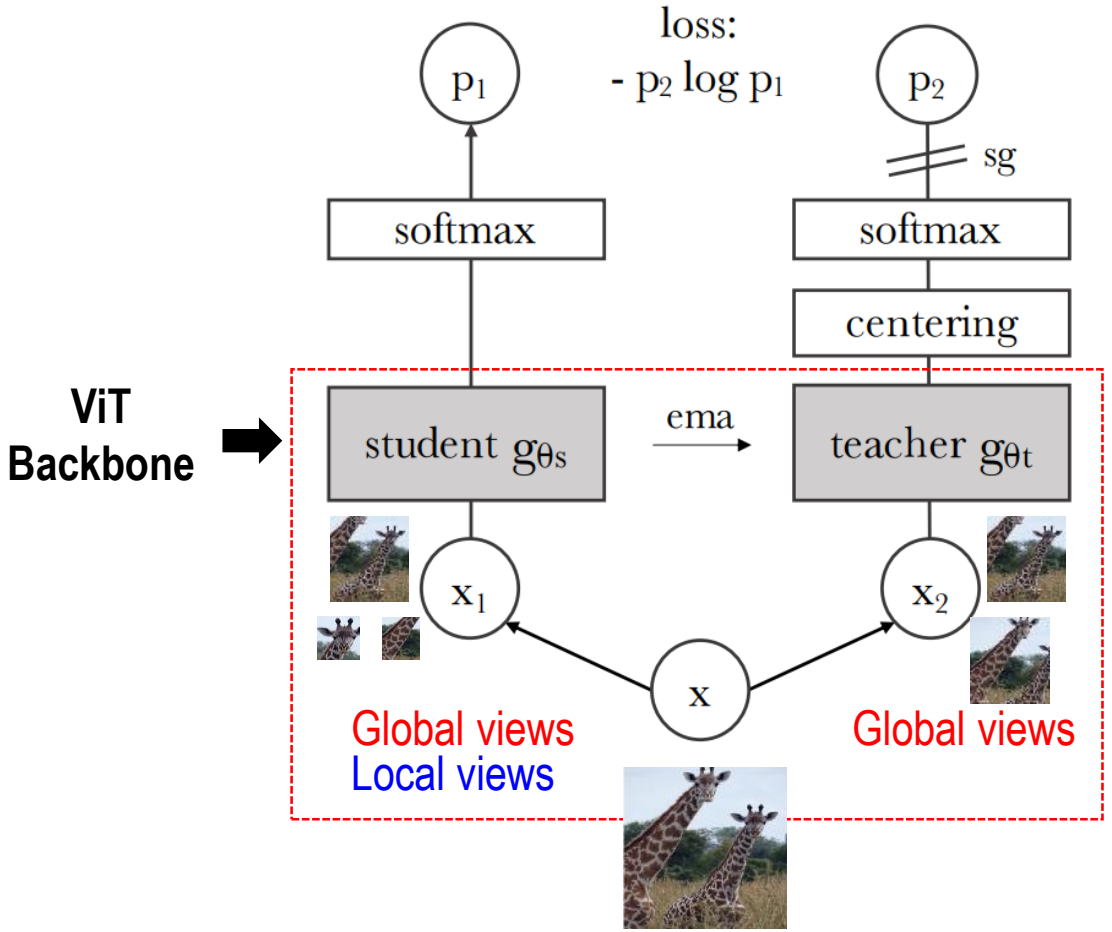
Figure 2: **Self-distillation with no labels.** We illustrate DINO in the case of one single pair of views (x_1, x_2) for simplicity. The model passes two different random transformations of an input image to the student and teacher networks. Both networks have the same architecture but different parameters. The output of the teacher network is centered with a mean computed over the batch. Each networks outputs a K dimensional feature that is normalized with a temperature softmax over the feature dimension. Their similarity is then measured with a cross-entropy loss. We apply a stop-gradient (sg) operator on the teacher to propagate gradients only through the student. The teacher parameters are updated with an exponential moving average (ema) of the student parameters.

Architecture (2/6)

- **Vision Transformer Backbone**
  - Both student and teacher use a ViT (e.g., ViT-S/16 or ViT-B/16). → Teacher is not stronger in terms of architecture

- **Multi-crop training**
  - Global views : Above 50% of image pixels → More information
  - Local views : Below 50% of image pixels





Architecture (3/6)

- **EMA (Exponential Moving Average)**
  - Update rule: m is close to 1 → slow moving average after every step

$$\theta_{\text{teacher}} \leftarrow m \cdot \theta_{\text{teacher}} + (1 - m) \cdot \theta_{\text{student}}$$

  - Why EMA?
    - Stability of the target distribution : consistent and stable supervision
    - Preventing collapse : slow update acts as an implicit regularizer
    - Better representation quality : reduce noise and produce robust feature
- **Centering**
  - Teacher produces a logit vector

$$z_{\text{teacher}} \in \mathbb{R}^K$$

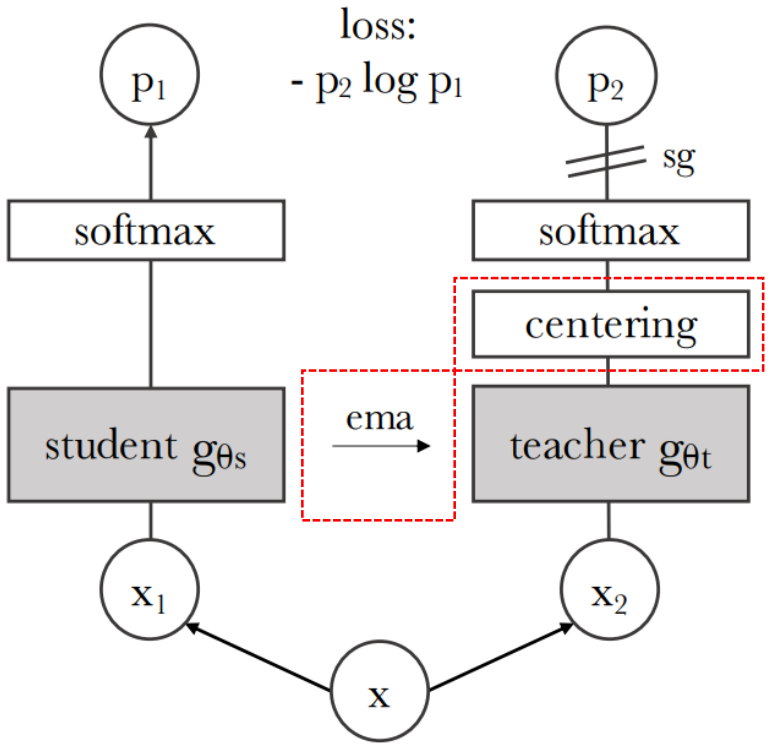
  - A running mean vector is maintained

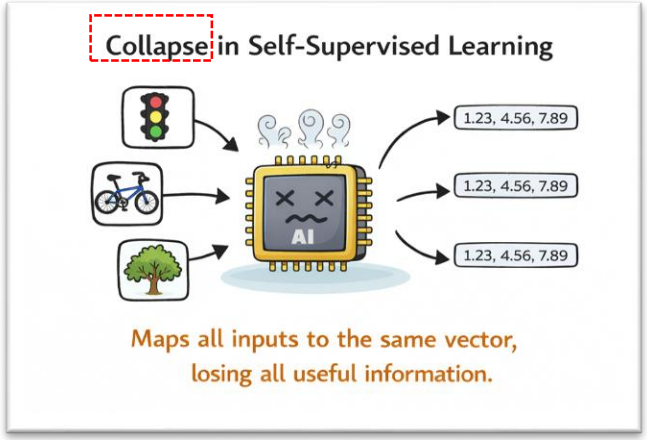
$$c \leftarrow \lambda c + (1 - \lambda) \frac{1}{B} \sum_{i=1}^{B} z_{\text{teacher}}^{(i)}$$

  - Subtracts this mean from teacher logits

$$\tilde{z}_{\text{teacher}} = z_{\text{teacher}} - c$$

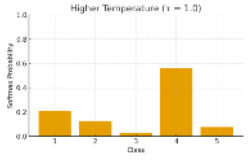
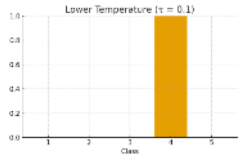
  - Why Centering?
    - forces the teacher's logits to stay zero-centered across dimensions
    - preventing drift and avoiding collapse when applying low-temperature softmax.





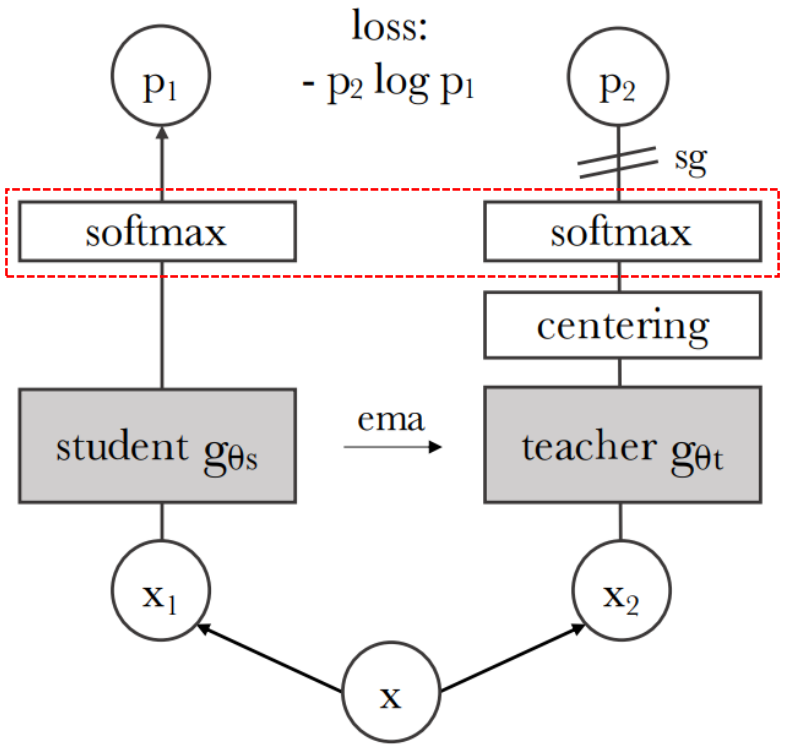
Architecture (4/6)

- **Difference of the Softmax**

	Student	Teacher
Temperature	Higher (e.g., 0.1) 	Low (e.g., 0.04) 
Distribution	Smooth, more spread out	Sharp, concentrated
Role	Avoiding collapse	High-confidence pseudo-labels
Effect	Increases training stability and prevents trivial constant predictions	Emphasizes the most informative, object-centric regions

Softmax with temperature (τ)

$$\sigma_i = \frac{\exp(z_i/\tau)}{\sum_j \exp(z_j/\tau)}$$



Architecture (5/6)

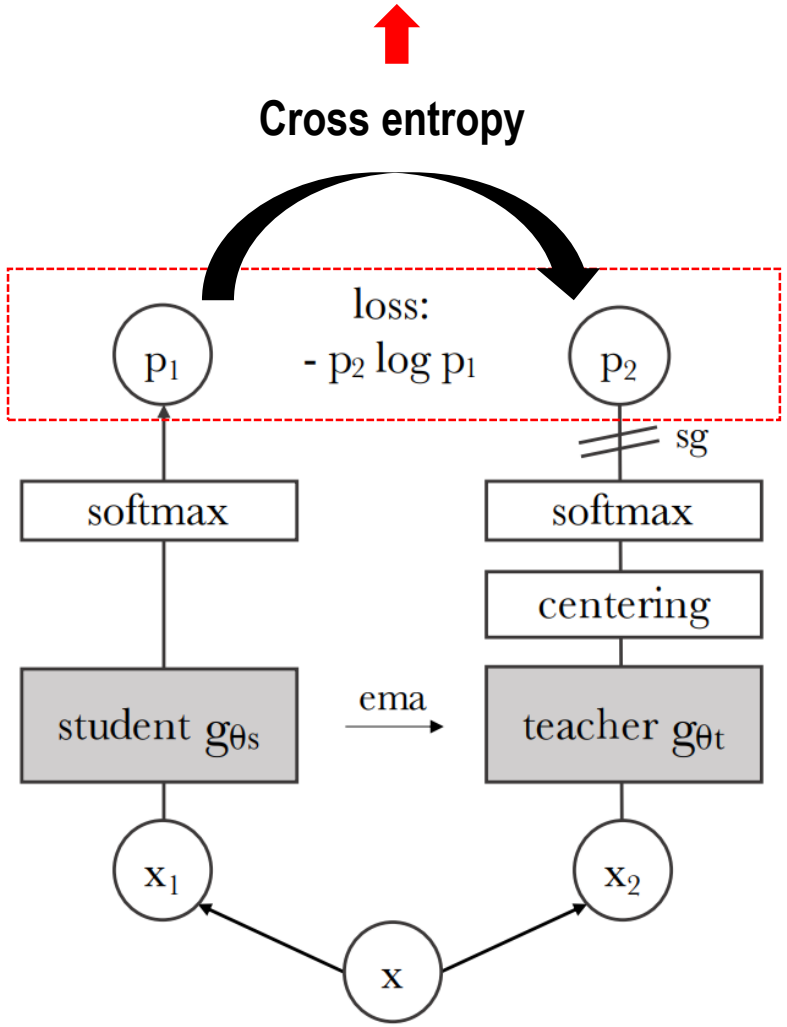
- **Output of ViT**
  - CLS token (384 dim) → MLP projection → Logit vector (65,536 dim)

- **Cross Entropy**

	Details
Meaning of the class	A semantic cluster discovered by teacher
Loss Function	Cross-Entropy between student output and teacher distribution
Why Cross Entropy ?	Efficiently aligns student with teacher's soft targets, even without labels

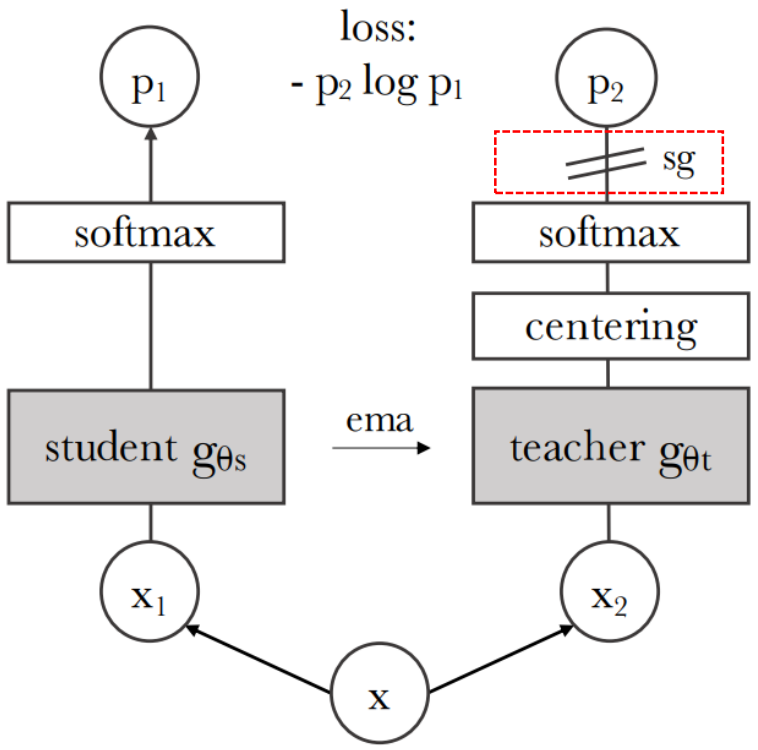
Learning local-to-global correspondence !

Labels in self-supervised learning ?

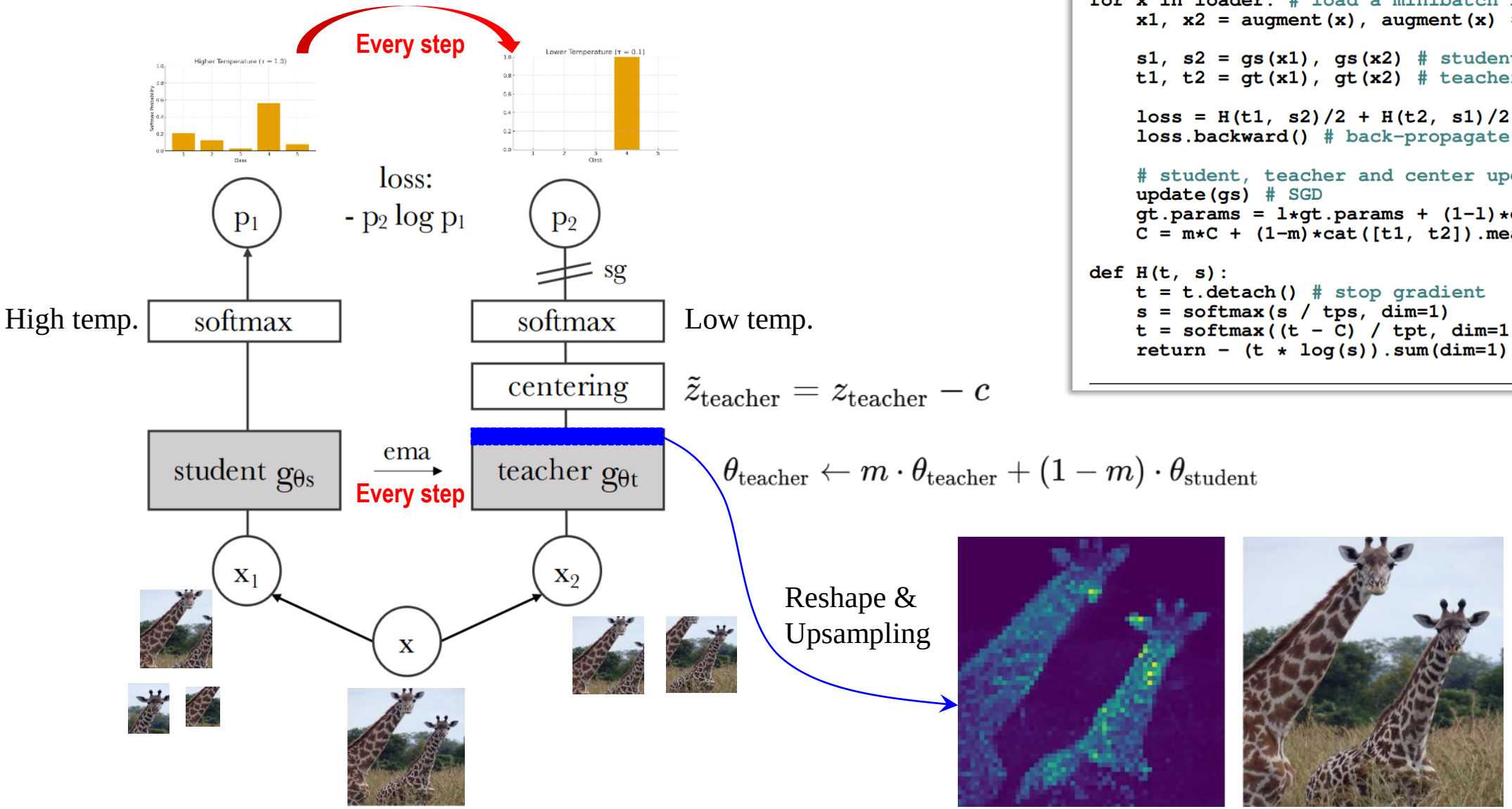


Architecture (6/6)

- **Stop gradient**
  - Teacher is updated only via EMA, not by backpropagation, maintaining consistent supervision.
  - Prevents gradients from flowing into the teacher network.



Architecture summary



Algorithm 1 DINO PyTorch pseudocode w/o multi-crop.

```
# gs, gt: student and teacher networks
# C: center (K)
# tps, tpt: student and teacher temperatures
# l, m: network and center momentum rates
gt.params = gs.params
for x in loader: # load a minibatch x with n samples
    x1, x2 = augment(x), augment(x) # random views

    s1, s2 = gs(x1), gs(x2) # student output n-by-K
    t1, t2 = gt(x1), gt(x2) # teacher output n-by-K

    loss = H(t1, s2)/2 + H(t2, s1)/2
    loss.backward() # back-propagate

    # student, teacher and center updates
    update(gs) # SGD
    gt.params = l*gt.params + (1-l)*gs.params
    C = m*C + (1-m)*cat([t1, t2]).mean(dim=0)

def H(t, s):
    t = t.detach() # stop gradient
    s = softmax(s / tps, dim=1)
    t = softmax((t - C) / tpt, dim=1) # center + sharpen
    return - (t * log(s)).sum(dim=1).mean()
```

Experiments

- Experiments

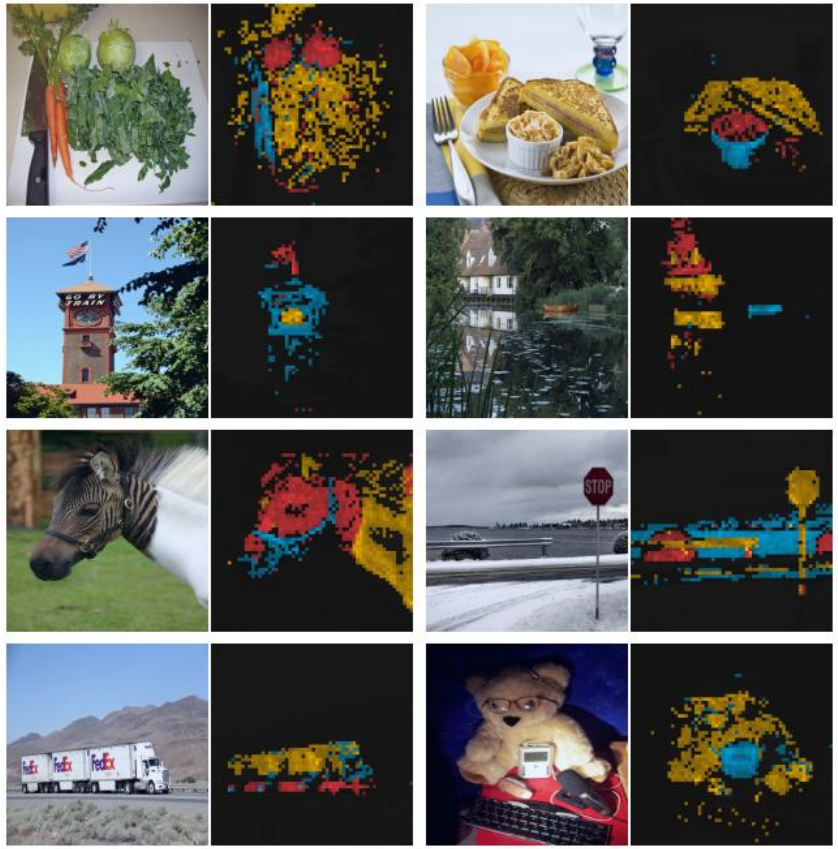
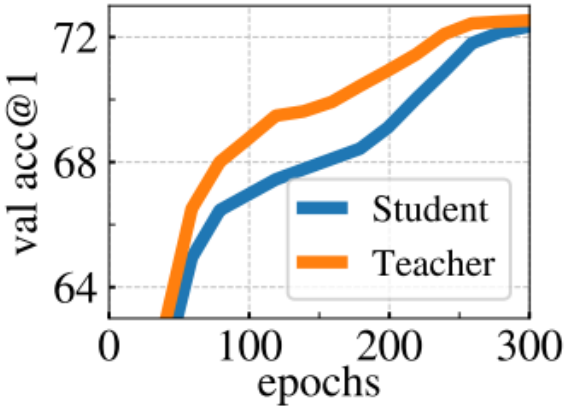


Figure 3: **Attention maps from multiple heads.** We consider the heads from the last layer of a ViT-S/8 trained with DINO and display the self-attention for `[CLS]` token query. Different heads, materialized by different colors, focus on different locations that represents different objects or parts (more examples in Appendix).



Teacher	Top-1
Student copy	0.1
Previous iter	0.1
Previous epoch	66.6
Momentum	72.8

Figure 6: Top-1 accuracy on ImageNet validation with k-NN classifier. **(left)** Comparison between the performance of the momentum teacher and the student during training. **(right)** Comparison between different types of teacher network. The momentum encoder leads to the best performance but is not the only viable option.